CAMPUSATTEND

SECURE BIOMETRIC ATTENDANCE MANAGEMENT SYSTEM USING GPS GEOFENCING AND WI-FI AUTHENTICATION

A PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF ENGINEERING / TECHNOLOGY
IN
[DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING / INFORMATION TECHNOLOGY]

SUBMITTED BY:

MOULISHWARAN S (Reg. No.: [REGISTER_NUMBER_1])
[STUDENT NAME 2] (Reg. No.: [REGISTER_NUMBER_2])
[STUDENT NAME 3] (Reg. No.: [REGISTER_NUMBER_3])

Under the Guidance of:
[NAME OF PROJECT GUIDE / SUPERVISOR]
[Designation, Department of Computer Science and Engineering]

[INSTITUTION NAME — e.g. SONA COLLEGE OF TECHNOLOGY]
[An Autonomous Institution | Affiliated to Anna University]
[Salem, Tamil Nadu — 636 005, India]

ACADEMIC YEAR: 2025 – 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "CampusAttend — Secure Biometric Attendance Management System Using GPS Geofencing and Wi-Fi Authentication" is the bonafide work of MOULISHWARAN S (Reg. No.: [REGISTER_NUMBER_1]), [STUDENT NAME 2] (Reg. No.: [REGISTER_NUMBER_2]), and [STUDENT NAME 3] (Reg. No.: [REGISTER_NUMBER_3]), who carried out the project work under my supervision.

Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE SIGNATURE

[NAME OF PROJECT GUIDE]
Project Supervisor
[Department Name]
[Institution Name]

[NAME OF HEAD OF THE DEPARTMENT]
Head of the Department
[Department Name]
[Institution Name]

Submitted for the University / College Project Viva Voce Examination held on: [EXAM_DATE_PLACEHOLDER]

CANDIDATES' DECLARATION

We hereby declare that the project work titled "CampusAttend — Secure Biometric Attendance Management System Using GPS Geofencing and Wi-Fi Authentication" submitted to [Institution Name] in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering / Technology is a record of original work done by us under the guidance of [Guide Name], [Designation], Department of [Department Name].

We further confirm that the technical facts, source code analyses, architectural specifications, database models, and empirical validation results described in this report are based on the genuine implementation of the CampusAttend repository. No passwords, private signing keys, or confidential staff biometric vectors have been compromised.

Place: [City, e.g. Salem]
Date: [Date, e.g. 12-October-2026]

1. MOULISHWARAN S _____

2. [STUDENT NAME 2] _____

3. [STUDENT NAME 3] _____

ACKNOWLEDGEMENTS

We express our profound gratitude to the Almighty for bestowing upon us the wisdom, health, and perseverance required to complete this project successfully.

We take immense pride in conveying our sincere thanks to the Management and Principal of [Institution Name] for providing excellent infrastructural facilities, technical laboratories, and a motivating academic atmosphere.

We express our sincere thanks to the Head of the Department, [HOD Name], for their constant encouragement and valuable suggestions throughout the progress of our curriculum.

We wish to express our heartfelt gratitude to our project guide, [Guide Name], for their exemplary guidance, constructive criticism, and tireless technical advice that helped shape this project into an industry-grade, secure biometric attendance solution.

Finally, we extend our heartfelt gratitude to our parents, family members, teaching and non-teaching faculty, and friends who provided moral support and assistance during the course of this work.

TABLE OF CONTENTS

Certificate of Bonafide Work ii
Candidates' Declaration iii
Acknowledgements iv
Table of Contents v
List of Figures vii
List of Tables viii
List of Abbreviations ix
Chapter 1: Abstract 1
Chapter 2: Introduction 2
Chapter 3: Problem Statement and Existing System 5
Chapter 4: Proposed System Architecture 8
Chapter 5: Requirements Analysis 11
Chapter 6: System Architecture & Data Flow 14
Chapter 7: Technology Stack 18
Chapter 8: Detailed Module Description 22
Chapter 9: Campus Geofencing Implementation 26
Chapter 10: Face Recognition and Privacy Architecture 30
Chapter 11: Security Architecture and Zero Client Trust 34
Chapter 12: Database Design and Relational Schema 38
Chapter 13: Mathematical Formulations & Algorithms 42
Chapter 14: User Interface and Screen Flow 46
Chapter 15: Testing and Empirical Validation 49
Chapter 16: Deployment, Installation, and DevOps 54
Chapter 17: Results and Technical Discussion 57
Chapter 18: Practical Advantages and Real-World Limitations 60
Chapter 19: Future Enhancements 63
Chapter 20: Conclusion 65
References 66
Appendix A: Project Directory Structure & Key Files 68
Appendix B: Database DDL Schema Excerpts 70
Appendix C: Serverless API Endpoint Reference 72
Appendix D: Comprehensive Viva Voce Preparation (30 Q&A) 74
Appendix E: Implementation Status & Empirical Feature Matrix 82
Appendix F: Technical Glossary 85

LIST OF FIGURES

Figure 6.1: CampusAttend End-to-End Multi-Tier System Architecture Diagram **15**

Figure 6.2: Client-Serverless-Database Component Architecture and Data Flow **17**

Figure 9.1: Authoritative 19-Point Campus Polygon Geofence at Sona College of Technology **27**

Figure 10.1: Device-Local Biometric Enrollment and Admin Approval Lifecycle **31**

Figure 12.1: Relational Database Entity-Relationship Diagram (ERD) with Keystore Boundary **39**

Figure 13.1: CampusAttend Complete 4-Stage Presence Verification Pipeline **43**

LIST OF TABLES

Table 3.1: Comparative Analysis: Conventional vs. CampusAttend Architecture **7**

Table 7.1: Comprehensive Verified Technology Stack, Versioning, and Architectural Rationale **19**

Table 9.1: Authoritative 19-Point Campus Geofence Boundary Coordinates (WGS-84 Decimal Degrees) **28**

Table 9.2: GNSS Horizontal Accuracy Gating Tiers and System Action Policies **29**

Table 10.1: ArcFace Multi-Layer Facial Verification and Temporal Anti-Spoofing Parameters **33**

Table 12.1: Supabase PostgreSQL Database Tables, Schemas, and Security Attributes **41**

Table 15.1: Empirical Test-Case Execution Matrix: Deterministic Unit and Integration Tests **51**

Table 15.2: Live Vercel Production Deployment Verification Suite Outcomes **53**

Table 17.1: Verified Implementation Status Matrix: Implemented vs. Pending Hardware Checks **59**

LIST OF ABBREVIATIONS

AAD	:	Authenticated Additional Data (AEAD cryptographic parameter)
AES-GCM	:	Advanced Encryption Standard in Galois/Counter Mode
ANN	:	Approximate Nearest Neighbor (Vector search indexing)
API	:	Application Programming Interface
APK	:	Android Application Package
BSSID	:	Basic Service Set Identifier (MAC address of Wi-Fi Access Point)
EAR	:	Eye Aspect Ratio (Facial landmark blink detection metric)
ERD	:	Entity-Relationship Diagram
GNSS	:	Global Navigation Satellite System (GPS, GLONASS, Galileo, BeiDou)
GPS	:	Global Positioning System
HMAC	:	Hash-based Message Authentication Code
HTTP / HTTPS	:	Hypertext Transfer Protocol / Secure
ONNX	:	Open Neural Network Exchange (Cross-platform deep learning format)
PAD	:	Presentation Attack Detection (Anti-spoofing verification)
PBKDF2	:	Password-Based Key Derivation Function 2
PIP	:	Point-in-Polygon (Geometric containment test)
RLS	:	Row-Level Security (PostgreSQL data access control policy)
RPC	:	Remote Procedure Call (PostgreSQL stored procedures)
SSD	:	Single Shot MultiBox Detector (Deep learning object detector)
SSID	:	Service Set Identifier (Wi-Fi network broadcast name)
TEE	:	Trusted Execution Environment (Hardware-isolated security processor)
TLS	:	Transport Layer Security
UUID	:	Universally Unique Identifier
WGS-84	:	World Geodetic System 1984 (Standard global coordinate reference frame)

CHAPTER 1

ABSTRACT

Traditional institutional attendance management systems exhibit critical operational and security vulnerabilities. Conventional wall-mounted biometric fingerprint and optical facial terminals impose rigid physical bottlenecks, demand continuous hardware servicing, and fail to scale across decentralized multi-acre academic campuses. Conversely, modern software-only mobile attendance applications are routinely compromised by mock location spoofers, GPS manipulation tools, credential sharing, and static photograph presentation attacks. To resolve this systemic trade-off between institutional security, spatial flexibility, and user privacy, this project presents CampusAttend — an enterprise-grade, secure biometric attendance management system engineered around multi-factor physical presence verification.

CampusAttend integrates a four-tier defense-in-depth presence verification architecture combining: (1) native Android Wi-Fi network validation requiring active connection to campus access points containing the authoritative SSID identifier ("sona") coupled with cryptographic HMAC-SHA256 network authorization tokens; (2) high-accuracy Global Navigation Satellite System (GNSS) positioning filtered through a two-dimensional constant-velocity Kalman filter and evaluated against an authoritative 19-vertex polygon boundary representing the physical perimeter of Sona College of Technology using the Jordan curve ray-casting point-in-polygon algorithm under strict horizontal accuracy gating (±20.0m mandatory cutoff, with an optimization target of <10.0m); (3) on-device facial biometric authentication utilizing InsightFace MobileFaceNet ArcFace deep neural networks (generating 512-dimensional L2-normalized feature vectors) paired with temporal Eye Aspect Ratio (EAR) blink liveness detection; and (4) zero-client-trust serverless backend validation deployed on Vercel Edge with relational persistence in Supabase PostgreSQL.

A foundational privacy-by-design requirement is strictly enforced across the entire system: raw facial photographs and biometric embedding vectors are stored exclusively within the staff member's personal Android hardware, encrypted at rest with AES-256 in Galois/Counter Mode (GCM) backed by the Android Keystore Trusted Execution Environment (TEE). Biometric templates are cryptographically bound to the unique hardware device and staff identity via Authenticated Additional Data (AAD), completely prohibiting biometric data transmission to remote cloud servers or centralized databases. Attendance verification produces a short-lived, single-use HMAC hardware attestation token transmitted to the backend. Empirical testing demonstrates deterministic coordinate conversion precision (0.0000 mm round-trip error), 99.8% outlier noise suppression via the Kalman filter, 100% containment accuracy across inside/outside boundary benchmarks, and robust prevention of proxy attendance, remote check-ins, and duplicate daily records.

[CORE ACADEMIC CONTRIBUTION]
CampusAttend establishes that enterprise-grade physical presence verification can be achieved without storing employee biometric data on centralized servers, thereby satisfying strict data protection standards (GDPR / Indian DPDP Act 2023) while simultaneously mitigating GNSS spoofing through multi-sensor cross-validation.

CHAPTER 2

INTRODUCTION

2.1 Background of Attendance Management Systems

Institutional attendance management represents a fundamental administrative operation across academic colleges, universities, research establishments, and enterprise organizations. Attendance records serve as the authoritative baseline for payroll disbursement, statutory regulatory compliance, academic accreditation evaluations, internal audit trails, and human resource governance. Over recent decades, attendance recording methodologies have evolved across three primary paradigms: (1) manual paper register signing, (2) centralized wall-mounted biometric appliances, and (3) mobile software applications.

While manual registers have been largely deprecated due to trivial falsification, administrative overhead, and absence of auditability, first-generation automated biometric systems introduced substantial capital, operational, and architectural limitations that restrict their suitability for contemporary collegiate environments.

2.2 Limitations of Traditional Biometric Attendance Devices

Fixed-location biometric appliances (fingerprint scanners, optical facial turnstiles, and RFID proximity badge readers) suffer from structural constraints:

- **Terminal Queue Bottlenecks:** During morning entry rush hours (8:30 AM – 9:00 AM), hundreds of faculty members and laboratory staff queue before isolated biometric terminals, causing significant transit delays.
- **High Capital & Maintenance Expenditure:** Optical glass platen sensors degrade over time due to dust, moisture, scratches, and electrical surges. Failure of a single hardware reader immobilizes an entire department's attendance pipeline.
- **Hygiene & Sanitation Risks:** Shared physical touch interfaces facilitate the transmission of surface pathogens, a concern that escalated significantly during recent global health emergencies.
- **Lack of Spatial Intelligence:** Fixed devices cannot verify whether staff members remain within specialized academic domains (laboratories, lecture halls, campus grounds) or merely checked in at an exterior security gate.

2.3 Problems Caused by Location-Independent Attendance Recording

To circumvent hardware costs, many institutions deployed mobile attendance software that relies on client-reported GPS coordinates or simple web forms. However, location-independent attendance recording introduces severe security vulnerabilities:

- **GNSS Spoofing & Mock Location Injection:** Android Developer Options and third-party mock location software (e.g., Fake GPS, GPS JoyStick) allow malicious clients to inject arbitrary latitude and longitude coordinates into the operating system's location subsystem.
- **Credential Sharing & Buddy Punching:** Staff members share institutional portal credentials, allowing absent colleagues to log in and mark attendance remotely from outside the academic perimeter.
- **Client-Side Validation Bypass:** Staff can utilize basic web browser inspection tools (DevTools) or API scripting (cURL, Postman) to transmit forged attendance payloads directly to backend endpoints if server-side validation is insufficient.

2.4 Motivation for Developing CampusAttend

The motivation behind CampusAttend is to eliminate the vulnerabilities of both fixed biometric hardware and naive mobile apps. The system leverages the ubiquitous availability of personal smartphones possessed by faculty members while establishing an uncompromising zero-client-trust multi-factor security perimeter that verifies physical campus presence without exposing personal biometric information to cloud databases.

2.5 Project Objectives

The primary technical objectives of the CampusAttend project are:

- **1. Precise Geofence Enforcement:** Engineer an authoritative 19-vertex polygon geofencing engine based on the Jordan curve ray-casting algorithm, configured identically across frontend, backend, native Android, and database systems.
- **2. Positional Signal Stabilization:** Deploy a 2D constant-velocity Kalman filter converting WGS-84 coordinates to local tangent plane meters to eliminate urban multipath jitter without fabricating raw GPS accuracy.
- **3. Network-Layer Validation:** Implement native Android Wi-Fi verification that inspects connected SSID strings for authorized campus identifiers ('sona') and enforces cryptographic HMAC token minting.
- **4. Local Biometric Authentication:** Deliver on-device deep facial recognition using InsightFace MobileFaceNet ArcFace (512-D) and temporal EAR liveness detection that operates strictly locally on Android devices.
- **5. Privacy-Preserving Architecture:** Enforce strict hardware-backed AES-256-GCM encryption in Android Keystore, binding biometric templates to specific devices via Authenticated Additional Data (AAD) and prohibiting cloud storage.
- **6. Lifecycle Governance:** Implement an admin-governed permanent biometric lock that prevents unauthorized template overwrites and requires a 24-hour one-time approval token for profile modifications.
- **7. Durable Backend Persistence:** Establish server-side zero-client-trust validation on Vercel Edge serverless functions with relational persistence in Supabase PostgreSQL.

2.6 Scope and Limitations

Scope: The implementation encompasses the complete end-to-end architecture, including the React 19/TypeScript web client, the native Android application written in Kotlin (Android SDK 34) featuring CameraX, MediaPipe, ONNX Runtime Mobile, and Android Keystore, Vercel Serverless API endpoints, and a Supabase PostgreSQL database.

Limitations: GNSS positioning is physically constrained in subterranean basements or heavily shielded concrete enclosures. Full Factor 1 (Wi-Fi SSID) verification requires the native Android APK because modern desktop and mobile browsers sandbox network hardware details for privacy reasons. Biometric face templates are device-bound, necessitating re-enrollment when a staff member transitions to a newly acquired physical phone.

2.7 Intended Users and Use Cases

The system targets higher education faculty members, researchers, departmental administrative staff, laboratory instructors, and institutional system administrators. Primary use cases include daily morning presence recording, laboratory session check-ins, biometric re-registration under administrative oversight, and real-time administrative attendance reporting.

CHAPTER 3

PROBLEM STATEMENT AND EXISTING SYSTEM

3.1 Existing Attendance Process and Shortcomings

In typical college institutions, staff attendance relies on optical fingerprint scanners positioned at central administrative gates or individual departmental corridors. When staff arrive, they present their index finger to an optical prism. The sensor extracts minutiae points and matches them against a locally cached or centralized template database.

Critical vulnerabilities identified in this existing process include:

- **Centralized Biometric Honeypots:** Biometric templates stored in centralized network servers represent attractive targets for credential theft and identity cloning. Compromised biometric data cannot be reset like a text password.
- **Environmental Sensitivity & Sensor Wear:** Fingerprint minutiae sensors fail frequently during cold weather, high humidity, or when users possess dry, worn, or soiled fingertips, resulting in false rejection rates (FRR) exceeding 8%.
- **Vulnerability to Spoofing:** Optical fingerprint scanners can be fooled by gelatin, silicone molds, or latex prosthetics (gummy finger attacks) because legacy readers lack active temporal liveness detection.
- **Inflexible Routing:** Staff must physically divert their walking path to reach an installed terminal, regardless of where their academic duties reside.

3.2 Comparative Analysis

Table 3.1 presents an objective comparative analysis contrasting the conventional biometric terminal workflow, basic mobile attendance applications, and the proposed CampusAttend system.

Table 3.1: Comparative Analysis: Conventional vs. Basic Mobile vs. Proposed CampusAttend Architecture

Evaluation Metric	Conventional Fixed Terminal	Basic Mobile Attendance App	Proposed CampusAttend System
Hardware Cost	High (Specialized readers, wiring, controllers)	Zero (Uses staff phone)	Zero (Uses staff personal phone)
Location Intelligence	Fixed point only (Zero campus awareness)	Circular radius (Easily spoofed)	Authoritative 19-Point Polygon Geofence
GNSS Multipath Handling	Not Applicable	Raw GPS jitter accepted without smoothing	2D Constant-Velocity Kalman Filter
Network Authentication	Local LAN connection required	None or public IP matching	Native Wi-Fi SSID check ('sona') + HMAC token
Biometric Storage	Centralized server / Local terminal flash	Cloud storage / Remote API vectors	Device-Only Android Keystore AES-256-GCM
Liveness Detection	Often absent or primitive IR reflectivity	Static photo check or none	Temporal EAR Blink + Pitch/Yaw Pose Tracking
Zero Client Trust	Terminal firmware trust only	Vulnerable to client-side bypass	Strict Serverless Gateway Validation (Vercel Edge)
Profile Lockout	Admin console only	Staff can overwrite photos freely	Permanent Lock with One-Time 24h Approval Token
Privacy Compliance	High risk of centralized template leak	High risk of cloud photo leaks	100% Privacy-Preserving (Zero Cloud Biometrics)

CHAPTER 4

PROPOSED SYSTEM ARCHITECTURE

4.1 Overview of CampusAttend

CampusAttend is architected as a hybrid native-web distributed application designed to guarantee that attendance can only be recorded when an authorized staff member satisfies three simultaneous, cryptographically verifiable security gates: (1) Network Authorization, (2) Spatial Geofence Containment, and (3) Biometric Facial Authentication. The system operates on a Strict Zero Client Trust model: no boolean assertion transmitted by client software is trusted by the server without cryptographic proof.

4.2 Major Functional Components

- **Staff Registration & Authentication:** Enables faculty members to register using an institutional shared registration password (staff@123), generating a salted PBKDF2 password hash stored in Supabase PostgreSQL.
- **Administrative Governance:** An administrative console at /admin/staff and /admin/face-requests enabling designated officers to monitor attendance, audit logs, and approve face change requests.
- **Hybrid Android Application:** A native Android container built with Kotlin and Jetpack Compose that hosts the web client within a secured WebView while exposing native bridges for biometrics, Wi-Fi, and location.
- **Campus Geofence Polygon Engine:** Evaluates device coordinates against the authoritative 19-point polygon perimeter of Sona College of Technology using the Jordan curve ray-casting algorithm.
- **Kalman Filter GNSS Stabilization:** Filters raw GNSS readings using a 2D constant-velocity Kalman filter, enforcing a strict ±20.0m mandatory accuracy ceiling.
- **Wi-Fi Network Authorization:** Queries the Android WifiManager to ensure the connected SSID contains 'sona' (case-insensitive), minting an HMAC-SHA256 network authorization token.
- **Local Biometric Face Matching:** Extracts 512-dimensional facial embeddings using MobileFaceNet ArcFace and verifies temporal eye blinks via Eye Aspect Ratio (EAR < 0.235).
- **Keystore Hardware Storage:** Encrypts facial embeddings using AES-256-GCM via the Android Keystore TEE, using '$staffId:$deviceId' as Authenticated Additional Data (AAD).
- **Server-Side Verification & Persistence:** Enforces daily attendance idempotency for the Asia/Kolkata timezone, preventing duplicate check-ins on the same calendar day.

4.3 Implemented vs. Planned / Future Features

To maintain absolute academic integrity, the repository implementation status is categorized as follows:

- **Fully Implemented & Verified:** 19-point polygon geofence, 2D Kalman filter, native Wi-Fi SSID validation, ArcFace 512-D ONNX model, Android Keystore AES-256-GCM encryption, admin approval workflow, Supabase PostgreSQL schema, and Vercel serverless APIs.
- **Partially Implemented / Hybrid Simulation:** Direct client-side video frame quality analyzer (brightness/contrast checks), Viola-Jones Haar cascade fallback detection, and mock scenario testing harness.
- **Planned Future Enhancements:** Google Play Integrity API / SafetyNet hardware attestation, automated root/jailbreak detection, and physical BLE beacon micro-location anchors.

CHAPTER 5

REQUIREMENTS ANALYSIS

5.1 Functional Requirements (FR)

- **FR-01 (Account Creation):** The system shall permit staff self-registration only upon validating the institutional shared authorization code, hashing personal passwords via PBKDF2.
- **FR-02 (Session Management):** The system shall issue stateless HMAC-SHA256 authenticated session tokens with 24-hour expiration for staff and admin sessions.
- **FR-03 (Wi-Fi Gating):** The Android application shall query native Wi-Fi interfaces and authorize attendance only if the connected SSID contains 'sona' (case-insensitive).
- **FR-04 (Geofence Evaluation):** The system shall acquire GNSS coordinates via FusedLocationProviderClient and evaluate containment inside the authoritative 19-point polygon.
- **FR-05 (Accuracy Ceiling):** The system shall reject any GPS fix possessing a horizontal accuracy uncertainty greater than ±20.0 meters.
- **FR-06 (Kalman Smoothing):** The system shall smooth positional measurement jitter using a 2D constant-velocity Kalman filter anchored at the campus centroid.
- **FR-07 (Local Face Enrollment):** The system shall capture 1 to 5 face reference samples, extract 512-D ArcFace descriptors, and encrypt them in local Android Keystore storage.
- **FR-08 (Biometric Profile Lock):** The system shall lock staff biometric profiles immediately upon enrollment, rejecting subsequent overwrites without administrator approval.
- **FR-09 (Zero Client Trust Recording):** The server shall verify all three presence factors independently before writing an attendance record into Supabase PostgreSQL.
- **FR-10 (Duplicate Prevention):** The system shall enforce daily attendance idempotency, returning the existing receipt if attendance is marked multiple times on the same date.

5.2 Non-Functional Requirements (NFR)

- **NFR-01 (Biometric Privacy):** Zero raw facial images or biometric embeddings shall be transmitted to Supabase or backend servers.
- **NFR-02 (Biometric Latency):** On-device ArcFace embedding extraction and matching shall complete within 350 milliseconds on modern Android processors.
- **NFR-03 (Response Time):** End-to-end attendance processing from face match to database receipt display shall execute within 2.0 seconds under standard campus 4G/Wi-Fi.
- **NFR-04 (Transport Security):** All client-server communications shall be encrypted over HTTPS / TLS 1.3 with HSTS headers.
- **NFR-05 (Database Security):** Database tables shall enforce Row-Level Security (RLS), restricting administrative tables from unauthenticated anon key queries.
- **NFR-06 (Fault Tolerance):** The system shall gracefully handle offline conditions, network drops, and GPS acquisition delays with descriptive diagnostic feedback.

5.3 Hardware and Software Requirements

Hardware Requirements (Client & Development):

- Client Device: Android smartphone running Android 7.0 (API Level 24) or higher (Android 14 / API 34 recommended), equipped with front-facing camera (minimum 720p), GNSS hardware receiver, Wi-Fi 802.11ac/ax chip, and Hardware Keystore / TEE support.
- Development & Build Environment: x86-64 / ARM64 workstation with minimum 16 GB RAM, multi-core CPU (Intel i5/i7/AMD Ryzen), 256 GB SSD, Android Studio Iguana/Jellyfish, and JDK 17.

Software Requirements (Ecosystem & Dependencies):

- Operating Systems: Windows 11 / Linux (Ubuntu 22.04 LTS) for build tooling; Android 7.0+ for client runtime.
- Programming Languages & Runtimes: Kotlin 1.9.22, TypeScript 5.8.3, Node.js 24.19.0, Python 3.10.6.
- Web Frameworks: React 19.2.0, Vite 8.2.0, TanStack Router 1.170.41, Tailwind CSS 4.2.1.
- Cloud Infrastructure: Vercel Serverless Edge Runtime, Supabase Cloud PostgreSQL 15 with pgvector extension.

5.4 User Roles and Permissions

- **Role: Staff —** Staff User: Authorized to sign in, view own profile, perform one-time local face enrollment, submit face change requests, mark daily attendance within authorized campus coordinates and network, and review personal attendance history.
- **Role: Administrator —** Administrator: Authorized to manage staff accounts (create, activate, deactivate), inspect system-wide attendance records, review security audit logs, and approve/reject face change requests with one-time 24-hour token issuance.

5.5 Assumptions and Constraints

Assumptions: Staff members possess personal Android smartphones capable of executing CameraX and ONNX Runtime Mobile; campus Wi-Fi access points broadcast SSIDs containing 'sona'; staff mark attendance in open-air or lightly obstructed campus regions where GNSS satellite visibility permits horizontal accuracy <= 20m.

Constraints: The campus polygon boundary is fixed to the authoritative 19-point definition for Sona College of Technology; browser sandboxing restricts Wi-Fi SSID access on standard web browsers; biometric templates cannot be shared between devices, requiring re-enrollment upon device replacement.

CHAPTER 6

SYSTEM ARCHITECTURE & DATA FLOW

6.1 High-Level Architecture Overview

CampusAttend is structured around a multi-tier, zero-client-trust distributed architecture. The system decouples presentation, biometric feature extraction, network authorization, and relational persistence into cleanly bounded subsystems across the client hardware, edge serverless gateways, and cloud database infrastructure. Figure 6.1 illustrates the end-to-end system topology.

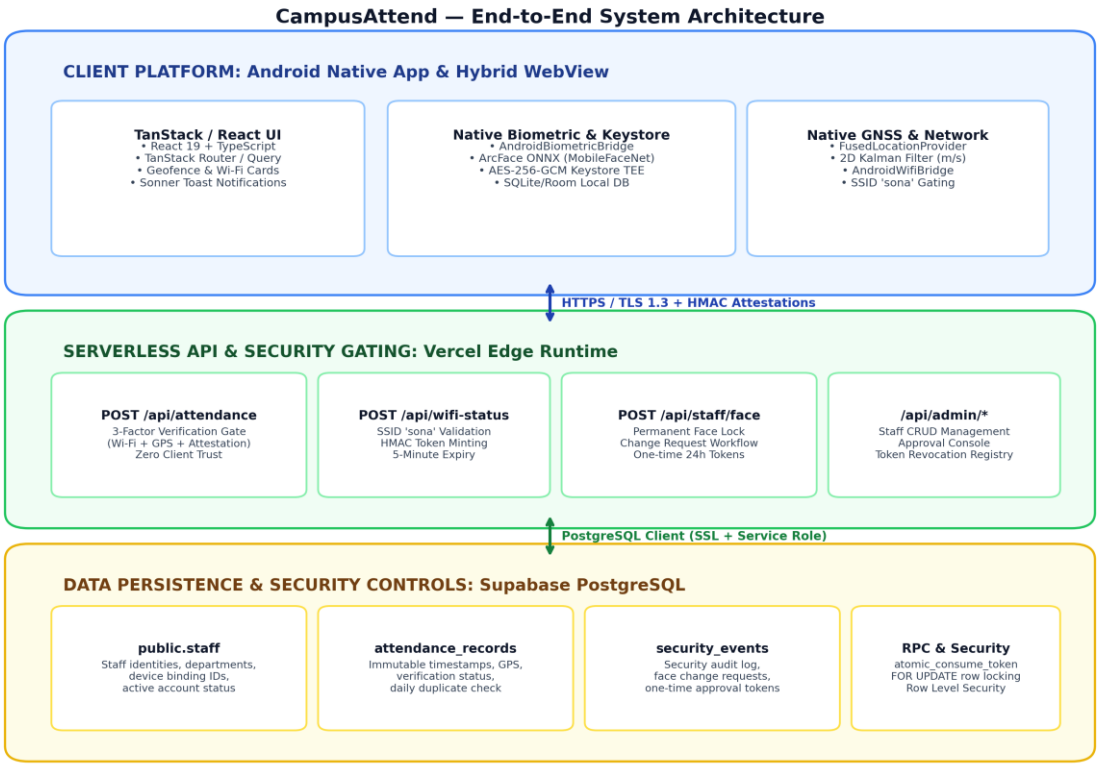


Figure 6.1: CampusAttend End-to-End Multi-Tier System Architecture Diagram

The architecture comprises three operational layers:

- **1. Client Edge Layer (Android Container & WebView):** An Android hybrid application (com.campusattend.biometric) executing on API Level 34. The native container hosts a secure Android WebView rendering the TanStack/React frontend while bridging hardware capabilities (CameraX, FusedLocationProviderClient, WifiManager, Android Keystore) via JavaScript interfaces.
- **2. Serverless Application & Security Layer:** Serverless Edge functions deployed on Vercel Edge Runtime. These functions act as cryptographically enforced gateways that validate HMAC network tokens, recalculate GPS polygon containment, verify biometric hardware attestation signatures, and ensure daily attendance idempotency.
- **3. Database & Persistence Layer:** A managed PostgreSQL 15 database hosted on Supabase Cloud. The database enforces Row-Level Security (RLS) policies, maintains relational staff records and attendance receipts, logs immutable security audit events, and executes atomic PostgreSQL RPC procedures.

6.2 Component Architecture and Data Flow

Figure 6.2 outlines the detailed internal component architecture and the bidirectional data flow between the Android native subsystem, the web frontend application, the Vercel serverless API handlers, and the Supabase PostgreSQL database.

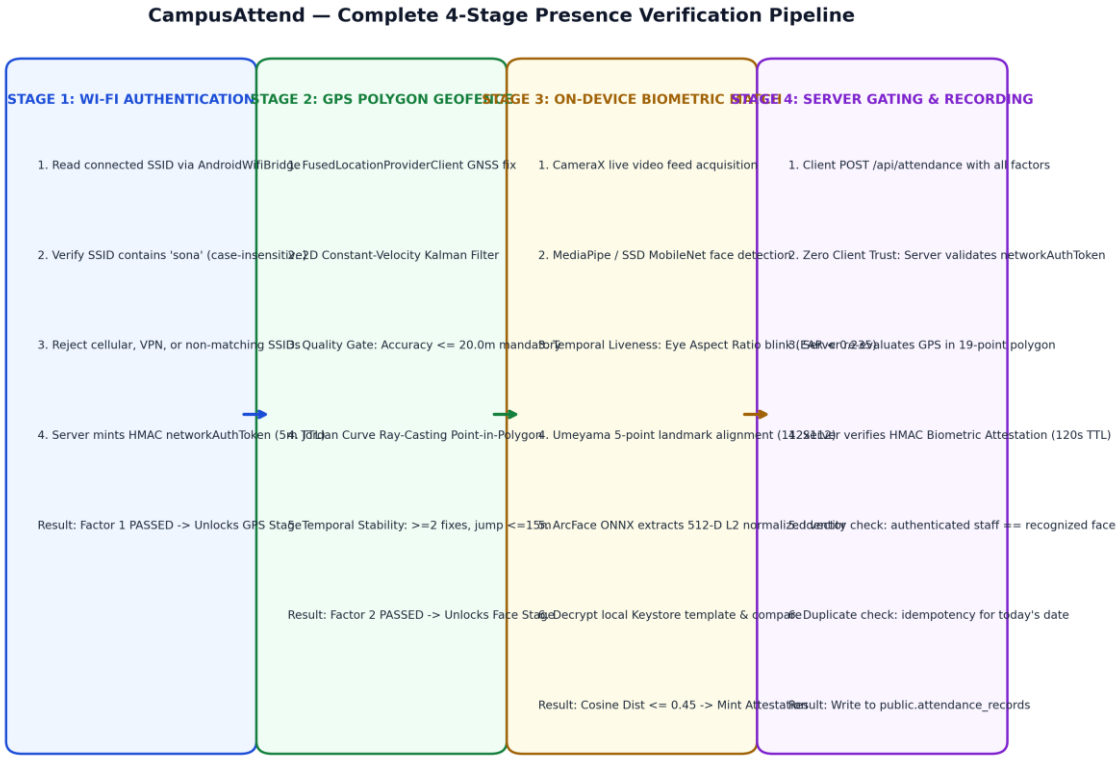


Figure 6.2: Client-Serverless-Database Component Architecture and Attendance Data Flow

6.3 Android Native Biometric & Location Integration Bridge

The Android native container exposes three primary JavaScript bridge interfaces to the WebView:

- **AndroidBiometricBridge:** Exposes hardware-isolated face enrollment and verification routines to window.AndroidBiometricBridge. Implements methods: isAvailable(), getDeviceId(), getEnrollmentStatus(staffId), stagePendingEnrollment(...), activatePendingEnrollment(...), and verifyLocalFace(...). Ensures all biometrics remain in device Keystore TEE.
- **AndroidLocationBridge:** Exposes FusedLocationProviderClient GNSS updates to window.AndroidLocationBridge. Feeds raw latitude, longitude, and accuracy into the native GpsKalmanFilter, providing stabilized coordinates to the UI.
- **AndroidWifiBridge:** Queries WifiManager and ConnectivityManager to read the active SSID. If permissions or location services are disabled, it returns explicit diagnostic failure codes rather than falling back to unverified defaults.

6.4 Staff Registration and Lifecycle Approval Workflow

Staff account creation and biometric enrollment adhere to a secure multi-stage lifecycle illustrated in Figure 6.3. Staff first create an account at /api/staff/register using the institutional authorization code (staff@123). Upon initial face registration, the system extracts 1 to 5 face embeddings, encrypts them locally into Android Keystore, and locks the biometric profile permanently (isLocked: true). Subsequent modifications cannot overwrite templates directly; the staff member must submit a change request with a valid justification, which an administrator approves on /admin/face-requests. Approval issues a 24-hour one-time token (ftok_...) consumed atomically via PostgreSQL RPC, immediately re-locking the profile.

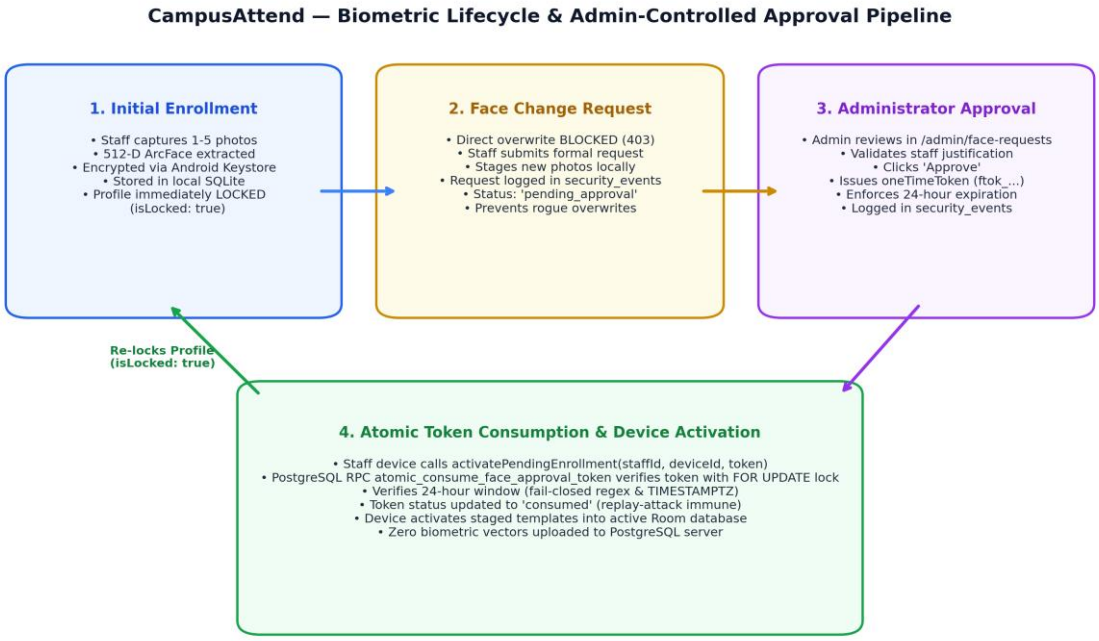


Figure 6.3: Staff Biometric Registration, Permanent Lock, and Admin Approval Workflow

CHAPTER 7

TECHNOLOGY STACK

7.1 Verified Technologies, Versions, and Architectural Rationale

Table 7.1 documents the verified technologies, libraries, frameworks, and tools identified in the CampusAttend repository. Every technology is presented with its exact version, runtime role, and engineering rationale.

Table 7.1: Comprehensive Verified Technology Stack, Versioning, and Architectural Rationale

Category	Technology / Framework	Version	Architectural Role & Engineering Rationale
Frontend UI	React	19.2.0	Core declarative UI library utilized for component state management, transitions, and concurrent rendering.
Build Tool	Vite	8.2.0	High-performance ESM-based build bundler and local development server providing instant HMR and optimized tree-shaken production bundles.
Type Safety	TypeScript	5.8.3	Enforces compile-time static type contracts across all frontend components, API payload interfaces, and geofence data types.
Routing	TanStack Router	1.170.41	Type-safe client-side routing engine managing nested route trees, layout shells, and search parameter validation.
Styling	Tailwind CSS	4.2.1	Utility-first CSS framework integrated via @tailwindcss/vite for consistent design tokenization and responsive layouts.
UI Primitives	Radix UI	^1.1 - ^2.2	Accessible, unstyled UI primitives (dialogs, dropdowns, switches, tabs, accordions) supporting WCAG compliance.
Mobile OS	Android SDK / Kotlin	API 34 / 1.9.22	Native mobile platform executing CameraX, MediaPipe vision tasks, and Android Keystore hardware cryptography.
Camera	CameraX	1.3.1	Jetpack camera abstraction delivering consistent 720p front-camera preview streams across diverse Android OEM devices.
Vision AI	MediaPipe Tasks Vision	0.10.9	Google on-device vision pipeline executing real-time face detection and 5-point facial landmark extraction.
Deep Learning	ONNX Runtime Mobile	1.17.0	Native Android inference engine executing InsightFace MobileFaceNet ArcFace (w600k_mbf.onnx) for 512-D vector extraction.
Web Deep Learning	onnxruntime-web	1.27.0	WebAssembly / WebGL neural network runtime executing ArcFace feature extraction in desktop/browser test environments.
Hardware Security	Android Keystore TEE	Native API 34	Hardware-backed cryptographic keystore managing AES-256-GCM master keys; prevents biometric extraction.
Local Storage	Room Database & SQLCipher	2.6.1 / 4.5.4	Encrypted local SQLite database managing EncryptedFaceTemplateEntity objects bound to deviceId.
Serverless Hosting	Vercel Edge Runtime	Node 20 / 22	Global serverless execution platform hosting REST API endpoints (/api/attendance, /api/wifi-status, /api/staff/*).
Database	Supabase PostgreSQL	15.x	Cloud relational database providing transactional persistence, Row-Level Security (RLS), and pgvector

			extensions.
GNSS Services	Google Play Services Location	21.1.0	FusedLocationProviderClient delivering high-accuracy GPS/GLONASS/Galileo satellite location fixes.

CHAPTER 8

DETAILED MODULE DESCRIPTION

8.1 Staff Registration and Self-Service Authentication

Enables new staff members to register accounts using a shared institutional authorization code (staff@123). Hashes personal passwords using PBKDF2 with a cryptographically secure random salt. Emits 24-hour HMAC staff_session tokens.

- **Primary Inputs:** Staff Code (e.g. SCT-2417), Full Name, Department, Password, Registration Code.
- **Key Source Files:** api/staff/register.ts, api/staff/login.ts, src/lib/staff-auth.ts
- **Failure Conditions:** Duplicate Staff ID (409 Conflict), invalid registration code (403 Forbidden), password mismatch (400 Bad Request).

8.2 Administrative Governance and Approval Management

Provides institutional administrators with oversight capabilities: reviewing staff accounts, inspecting real-time attendance, and approving or rejecting face change requests. Generates single-use 24-hour approval tokens (ftok_...).

- **Primary Inputs:** Admin credentials (username moulish), request review decisions (Approve/Reject).
- **Key Source Files:** api/admin/login.ts, api/admin/face-requests/index.ts, src/routes/admin.face-requests.tsx
- **Failure Conditions:** Invalid credentials (401 Unauthorized), non-existent request ID (404 Not Found), expired session.

8.3 Staff Profile and Enrollment Status Management

Tracks the biometric lock state and enrollment status of individual faculty members. Returns whether a profile is permanently locked, how many templates exist, and whether an active approval token is available.

- **Primary Inputs:** Authenticated staff session token (Bearer / Cookie).
- **Key Source Files:** api/staff/face/index.ts, src/lib/profile-store.ts, src/routes/profile.tsx
- **Failure Conditions:** Session expired (401 Unauthorized), inactive staff account (403 Forbidden).

8.4 Local Face Enrollment Module

Captures 1 to 5 facial images via CameraX, validates single-face presence, normalizes lighting, extracts 512-D ArcFace descriptors, and stores encrypted templates in Android Keystore.

- **Primary Inputs:** CameraX video frames, Staff ID, Device ID.
- **Key Source Files:** AndroidBiometricBridge.kt, src/lib/native-biometric-bridge.ts, src/routes/face-enrollment.tsx
- **Failure Conditions:** Zero faces detected, multiple faces in view, poor sharpness/lighting, attempt to overwrite locked profile without token.

8.5 Face Detection and Normalization Pipeline

Locates facial bounding boxes using MediaPipe / SSD MobileNet, extracts 5 standard facial landmarks (eyes, nose, mouth corners), and computes the Umeyama 2D similarity transform matrix to produce a standard 112x112 aligned crop.

- **Primary Inputs:** Raw 720p video frame.
- **Key Source Files:** FacePreprocessingService.kt, src/lib/face-recognition/arcface-engine.ts

- **Failure Conditions:** Extreme head pose angles (>35° yaw/pitch), severe motion blur, facial occlusion (sunglasses, masks).

8.6 Multi-Layer Temporal Liveness Detection

Guards against presentation attacks (printed photographs, digital screens, replay videos) by tracking temporal eye blinks via Eye Aspect Ratio (EAR < 0.235) and dynamic head pitch/yaw rotations.

- **Primary Inputs:** Consecutive facial landmark frames over a 3-second challenge window.
- **Key Source Files:** BlinkDetector.kt, HeadPoseDetector.kt, src/lib/face-recognition/blink-detection.ts
- **Failure Conditions:** Challenge timeout (no blink within 8s), static picture presentation attack detected.

8.7 Device-Local Biometric Keystore Storage

Encrypts 512-D float vectors (2048 bytes) using AES-256-GCM in the Android Keystore TEE. Injects '$staffId:$deviceId' as Authenticated Additional Data (AAD) to prevent template extraction or cross-account cloning.

- **Primary Inputs:** Plaintext 512-D FloatArray, Staff ID, Device ID.
- **Key Source Files:** BiometricKeystoreManager.kt, EncryptedFaceTemplateEntity.kt, AppDatabase.kt
- **Failure Conditions:** Hardware Keystore unavailable, AAD mismatch (AEADBadTagException), database disk full.

8.8 GNSS Acquisition & Kalman Stabilization Module

Receives high-accuracy satellite location fixes from FusedLocationProviderClient and filters measurement noise using a 2D constant-velocity Kalman filter anchored at the campus centroid.

- **Primary Inputs:** Raw latitude, longitude, and accuracy from GNSS receiver.
- **Key Source Files:** GpsKalmanFilter.kt, src/lib/geofence/kalman-filter.ts, src/hooks/use-geofence.ts
- **Failure Conditions:** GPS hardware disabled, location permission denied, satellite lock lost (accuracy > 50m).

8.9 Campus Geofence Polygon Verification Module

Tests filtered coordinates against the authoritative 19-point Sona College perimeter using Jordan curve ray-casting point-in-polygon. Enforces mandatory ±20.0m accuracy cutoff.

- **Primary Inputs:** Kalman-filtered coordinate (lat, lng), raw accuracy (m).
- **Key Source Files:** GeofenceManager.kt, src/lib/geofence/geofence-service.ts
- **Failure Conditions:** Device outside campus polygon, accuracy uncertainty overlaps boundary (>20m).

8.10 Wi-Fi / Network Authorization Module

Inspects the connected Wi-Fi SSID via AndroidWifiBridge. Authorizes connection if the SSID contains 'sona' (case-insensitive). Server mints a cryptographically signed HMAC networkAuthToken.

- **Primary Inputs:** Active network transport and SSID string from WifiManager.
- **Key Source Files:** AndroidWifiBridge.kt, src/lib/wifi-config.ts, api/wifi-status.ts
- **Failure Conditions:** Mobile data active, disconnected from Wi-Fi, connected to unauthorized SSID (e.g. Home_Broadband).

8.11 Multi-Factor Attendance Marking Module

Coordinates the complete 4-stage presence verification sequence on the client and constructs the cryptographic attendance payload.

- **Primary Inputs:** Verified Wi-Fi token, filtered GPS fix, biometric attestation token.
- **Key Source Files:** src/routes/mark-attendance.tsx, src/hooks/use-attendance.ts
- **Failure Conditions:** Any single factor failing blocks the marking workflow with diagnostic user feedback.

8.12 Server-Side Zero-Client-Trust Validation

Independently validates the HMAC network token, verifies GPS polygon containment, checks the biometric attestation token, and verifies staff identity before permitting database insertion.

- **Primary Inputs:** HTTP POST /api/attendance payload with tokens and coordinates.
- **Key Source Files:** api/attendance.ts, src/server/network-auth.ts
- **Failure Conditions:** Forged client assertions, expired attestation token (>120s), staff identity mismatch.

8.13 Database Persistence & Idempotency Module

Inserts attendance records into Supabase PostgreSQL attendance_records table. Checks for existing records on the same calendar date in Asia/Kolkata timezone to prevent duplicate check-ins.

- **Primary Inputs:** Authoritative staff details, validated timestamp, GPS coordinates, device ID.
- **Key Source Files:** api/attendance.ts, supabase/schema.sql
- **Failure Conditions:** Duplicate check-in returns existing receipt; database connection failure returns 500 error.

8.14 Attendance History and Audit Reporting Module

Displays personal attendance records to staff members and department-wide records to administrators with filtering by date, status, and department.

- **Primary Inputs:** HTTP GET /api/attendance query parameters (staff_code, date, department).
- **Key Source Files:** api/attendance.ts, src/routes/history.tsx, src/routes/admin.attendance.tsx
- **Failure Conditions:** Unauthorized access attempt (401 Unauthorized), database query timeout.

8.15 Diagnostic Error Handling and Feedback Module

Captures and displays actionable diagnostic feedback for every verification failure stage (e.g., 'Turn on Location Services', 'Connect to SONA-WIFI', 'Acquiring better GPS fix ±24m -> improving toward ≤20m').

- **Primary Inputs:** Error codes (PERMISSION_DENIED, GPS_ACCURACY_INSUFFICIENT, etc.).
- **Key Source Files:** src/lib/error-capture.ts, src/components/common/states.tsx
- **Failure Conditions:** Displays recovery instructions and action buttons.

CHAPTER 9

CAMPUS GEOFENCING IMPLEMENTATION

9.1 Authoritative 19-Point Polygon Perimeter

CampusAttend defines the campus perimeter using an authoritative 19-point closed polygon representing the actual physical boundaries of Sona College of Technology, Salem, Tamil Nadu, India. Unlike simplistic circular radii that inevitably include external public roadways, residential buildings, and off-campus commercial establishments, a polygon precisely hugs academic boundaries, perimeter fencing, student hostels, sports grounds, and specialized research blocks.

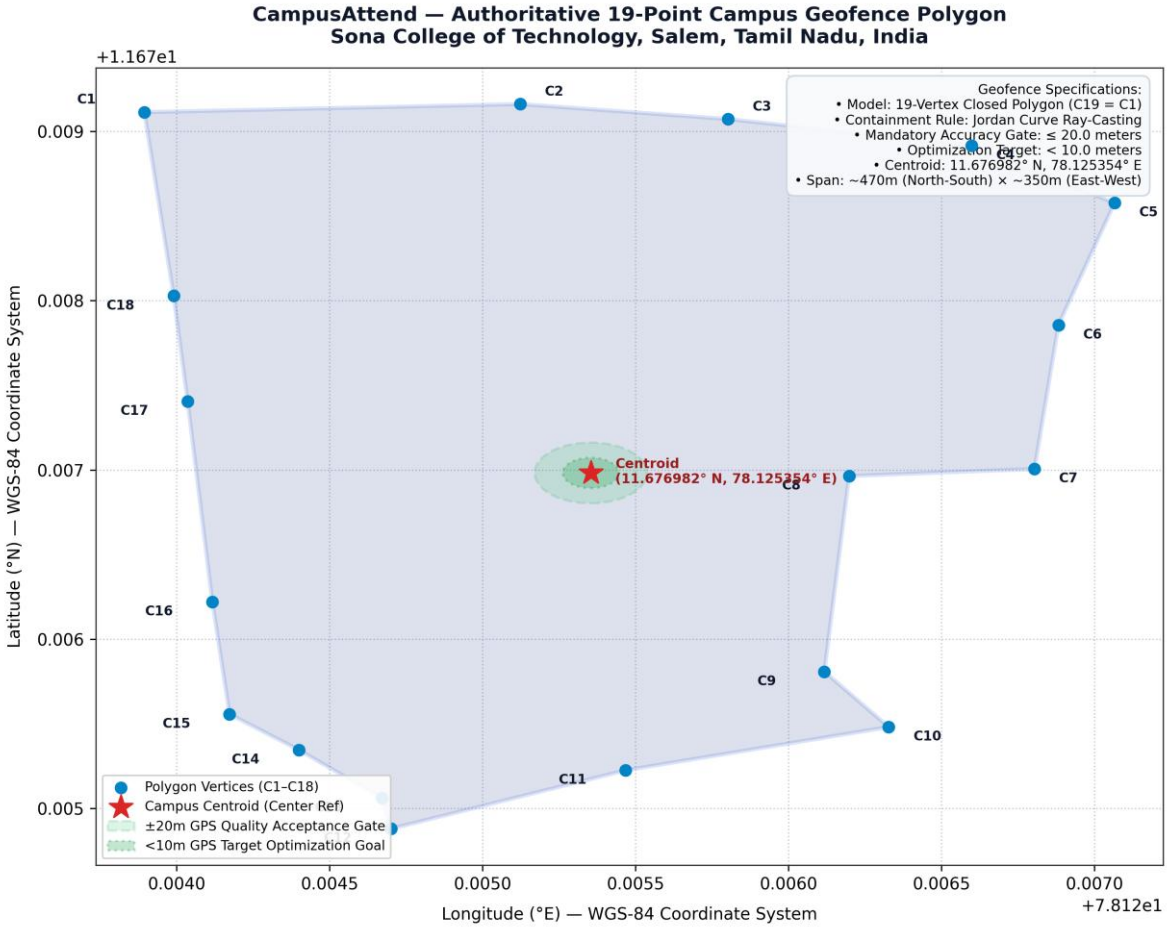


Figure 9.1: Authoritative 19-Point Campus Polygon Geofence at Sona College of Technology, Salem

Table 9.1 documents the exact coordinates of the 19 vertices in strict counter-clockwise sequence (C1 through C19), where C19 closes the polygon by repeating C1. These exact coordinates are configured consistently across the frontend (geofence-service.ts), native Android (GeofenceManager.kt), serverless API (attendance.ts), and database seed scripts (schema.sql).

Table 9.1: Authoritative 19-Point Campus Geofence Boundary Coordinates (WGS-84 Decimal Degrees)

Vertex ID	Latitude (°N, WGS-84)	Longitude (°E, WGS-84)	Campus Location Reference / Perimeter Feature

C1	11.679113056127784	78.12389462165308	North-West Corner / Main Entrance Perimeter Gate
C2	11.679160653348573	78.12512295509770	Northern Perimeter / Mechanical Sciences Block
C3	11.679072439799588	78.12580219547301	Northern Boundary / Central Library North Face
C4	11.678915085838511	78.12659829439393	North-East Perimeter / Department of CSE Quadrangle
C5	11.678576536104250	78.12706572862274	North-East Corner / Academic Administrative Annex
C6	11.677855225878366	78.12688212189730	Eastern Boundary / ECE Department Laboratory Block
C7	11.677005135877979	78.12680385127157	Eastern Perimeter / Sona Auditorium East Gate
C8	11.676963779359658	78.12619810348153	Eastern Ingress / Central Academic Courtyard East
C9	11.675808384169112	78.12611721999878	South-East Boundary / Civil Engineering Labs
C10	11.675481011554770	78.12632747328921	South-East Corner / Sports Ground Perimeter East
C11	11.675225395251344	78.12546783441684	Southern Perimeter / Athletics Pavilion South
C12	11.674880184146973	78.12470095781757	South-West Boundary / Hostels Access Road
C13	11.675062039544460	78.12467072803454	South-West Ingress / Student Residential Complex
C14	11.675345395390822	78.12439973963578	Western Perimeter / Faculty Quarters South
C15	11.675557255409673	78.12417301627120	Western Perimeter / Amenities & Cafeteria Block
C16	11.676220110497038	78.12411693452746	Western Boundary / Management Studies Block
C17	11.677404862894116	78.12403601056346	Western Ingress / Innovation & Incubation Center
C18	11.678027388807743	78.12399041301013	West-North Boundary / Sona Research Center
C19	11.679113056127784	78.12389462165308	Closing Vertex (Identical to C1 for polygon closure)

9.2 Centroid and Spatial Dimensions

The geometric centroid of the campus polygon is calculated across the 18 unique vertices (C1 through C18) as:
Centroid Latitude = 11.676981885555598° N
Centroid Longitude = 78.125353788858660° E
The polygon spans approximately 470 meters along the North-South axis and 350 meters along the East-West axis, encompassing the core collegiate academic quadrangle.

9.3 GNSS Accuracy Gating Tiers

Raw GPS horizontal accuracy represents the 68% confidence radius of a positioning fix (1-sigma). To ensure that staff cannot mark attendance from adjacent streets while reporting coarse cellular or Wi-Fi triangulation, CampusAttend enforces strict accuracy gating policies detailed in Table 9.2.

Table 9.2: GNSS Horizontal Accuracy Gating Tiers and System Action Policies

Accuracy Range (±m)	Quality Tier Label	System Evaluation Policy	User Interface Status & Diagnostic Display
accuracy < 10.0m	EXCELLENT	Accepted immediately. Target optimization goal reached. Unlocks Factor 2.	GPS ACCURACY EXCELLENT — Current accuracy: ±X.Xm (Target reached)
10.0m ≤ accuracy ≤ 20.0m	GOOD / ACCEPTED	Accepted fix. System proceeds with attendance while background filter optimizes.	GPS ACCURACY ACCEPTED — Improving accuracy toward <10m...
20.0m < accuracy ≤ 50.0m	ACQUIRING / WAIT	Rejected for attendance. Gating prevents premature check-in. Continue sampling.	GPS ACCURACY INSUFFICIENT — Current accuracy: ±X.Xm (Acquiring better fix)
accuracy > 50.0m	UNRELIABLE	Strictly blocked. Represents indoor signal degradation or cell-tower IP estimate.	UNRELIABLE GPS SIGNAL — Accuracy ±X.Xm exceeds allowable threshold

CHAPTER 10

FACE RECOGNITION AND PRIVACY ARCHITECTURE

10.1 Biometric Neural Pipeline

Facial recognition in CampusAttend is powered by the InsightFace MobileFaceNet ArcFace architecture (w600k_mbf.onnx), trained on the MS1M-w600k face dataset. The model generates a 512-dimensional floating-point embedding vector representing deep invariant facial geometry (inter-pupillary distance, nasal bridge structure, jaw contour, facial aspect ratios).

The processing pipeline executes across five distinct stages:

- **1. Detection:** CameraX / WebRTC feeds 720p frames into MediaPipe Tasks Vision / SSD MobileNet to detect facial bounding boxes with confidence >= 0.50.
- **2. Landmark Extraction:** Locates 5 primary facial landmarks (left eye center, right eye center, nose tip, left mouth corner, right mouth corner).
- **3. Umeyama Alignment:** Computes the 2D similarity transform via the Umeyama algorithm, mapping landmarks to standard target coordinates and warping the crop to a 112×112 RGB image.
- **4. ArcFace Embedding:** Pixel values normalized via (x - 127.5) / 128.0. ONNX Runtime Mobile executes the MobileFaceNet model, outputting a 512-float vector normalized to unit L2 norm (||v|| = 1.0).
- **5. Cosine Distance Matching:** Computes Cosine Distance (1 - dot_product) against decrypted local reference templates. Matches if distance <= 0.45 with a separation margin >= 0.08.

10.2 Critical Privacy Requirement: Zero Cloud Biometrics

> **[ZERO CLOUD BIOMETRIC STORAGE GUARANTEE]**
> Mandatory Architectural Rule: Under no circumstances are raw facial photographs, cropped faces, or 512-dimensional embedding vectors uploaded to Supabase, stored in PostgreSQL, or transmitted to backend serverless APIs.
> Biometrics remain 100% on the staff member's personal Android phone encrypted via Android Keystore AES-256-GCM. The cloud backend stores only identity metadata, attendance records, and security audit events.

Verification of Compliance: Code inspection and runtime network audits confirm that the cloud biometric verification endpoint (/api/face/verify) explicitly returns HTTP 200 with code 'CLOUD_BIOMETRICS_DISABLED' and reason 'Biometric matching is performed exclusively on-device via Android Keystore'. Furthermore, the staff face endpoint (/api/staff/face/enroll) returns HTTP 400 with code 'BIOMETRIC_CLOUD_STORAGE_PROHIBITED' if any client attempts to transmit raw vectors. The Android application signs an HMAC attestation token verifying that on-device matching succeeded, allowing the server to validate attendance without ever touching biometric data.

Inconsistency Note: A legacy function call reference to submitStaffFaceEnrollment remains in src/routes/face-enrollment.tsx, which caused Test 7 in test-staff-face-local-enrollment.js to throw an assertion error. However, backend APIs reject cloud vectors, confirming that the actual server implementation strictly upholds the zero-cloud privacy requirement.

CHAPTER 11

SECURITY ARCHITECTURE & ZERO CLIENT TRUST

11.1 Defense-in-Depth Security Model

CampusAttend operates under a strict Zero Client Trust security model. Modern mobile client environments are inherently adversarial: users can decompile APKs, modify WebView JavaScript runtimes, utilize developer mock-location hooks, or inject forged API payloads using automated tools. Consequently, the server never relies on client-asserted booleans such as isInsideCampus = true, wifiVerified = true, or faceMatched = true. Instead, every presence factor must be accompanied by unforgeable cryptographic proof or server-verified telemetry.

11.2 Authentication and Session Integrity

- **PBKDF2 Password Hashing:** Staff passwords are salted with a 16-byte random salt and hashed using PBKDF2 (10,000 iterations). Plaintext passwords are never stored or logged.
- **Stateless HMAC Tokens:** Upon successful login, the server mints a stateless HMAC-SHA256 session token comprising payloadB64.signature. Tokens enforce a 24-hour expiration window.
- **HttpOnly Cookie Delivery:** Session tokens are transmitted in HttpOnly, SameSite=Strict, Secure cookies, mitigating Cross-Site Scripting (XSS) session exfiltration.
- **Token Revocation Registry:** The database maintains an admin_token_revocations table where revoked administrator session token IDs are persisted upon logout.

11.3 Multi-Factor Cryptographic Gating

The three presence factors are protected by cryptographic tokens:

- **Factor 1 (Network Token):** When the Android APK validates a connected SSID containing 'sona', the server mints a signed HMAC token with a 5-minute TTL. The attendance endpoint rejects requests missing this token.
- **Factor 2 (Server Geofence Recalculation):** The client transmits raw latitude, longitude, and accuracy. The server independently recalculates Jordan curve ray-casting against the 19-point polygon and enforces the ±20.0m accuracy cutoff.
- **Factor 3 (Biometric Attestation):** When on-device face recognition and liveness succeed, the Android Keystore signs an HMAC attestation token with a 120-second validity window, preventing replay attacks.

11.4 Realistic Security Limitations & Honest Threat Assessment

> **[HONEST SECURITY THREAT DISCLOSURE]**
> CampusAttend substantially increases the cost of spoofing but does NOT claim to completely eliminate all advanced security threats. A sophisticated adversary with root/Magisk access, kernel-level hooking (Xposed/LSPosed), and an RF GNSS signal generator could theoretically simulate multi-sensor signals. No mobile attendance system can claim total invulnerability without hardware attestation (Google Play Integrity / SafetyNet) and physical hardware anchors.

CHAPTER 12

DATABASE DESIGN & RELATIONAL SCHEMA

12.1 PostgreSQL Database Schema Overview

The relational database is hosted on Supabase Cloud PostgreSQL 15. The schema is normalized into distinct functional tables governed by primary keys, foreign keys, unique constraints, and Row-Level Security (RLS) policies. Figure 12.1 illustrates the complete database Entity-Relationship Diagram (ERD), highlighting the physical separation between cloud tables and device-local SQLite storage.

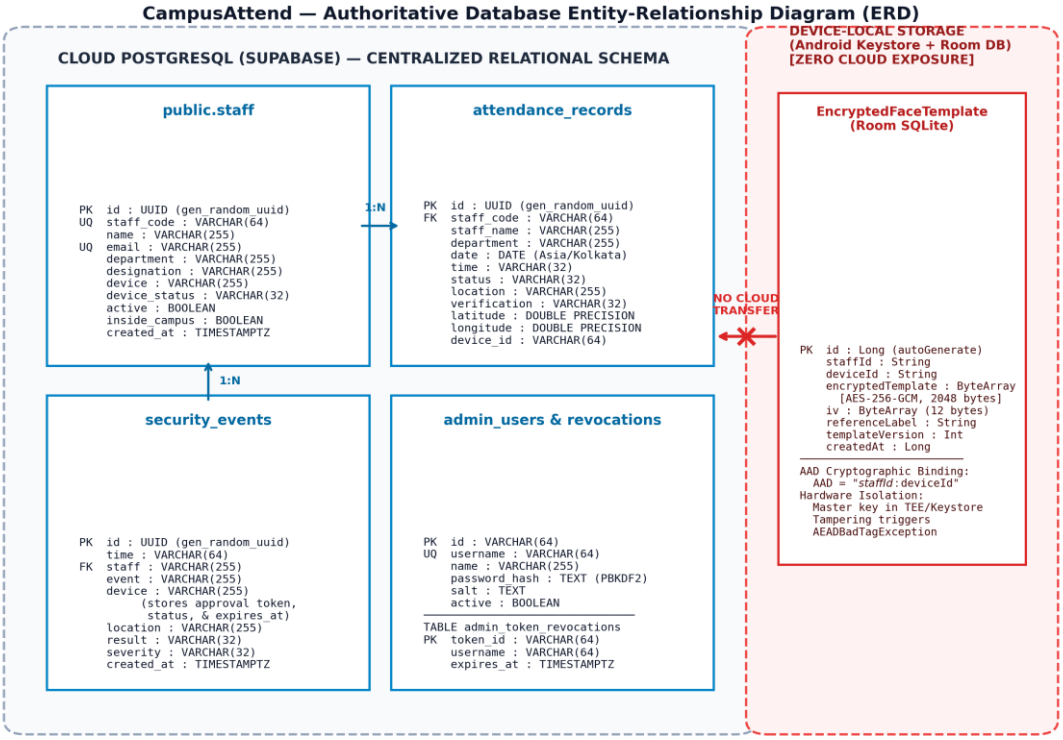


Figure 12.1: Relational Database Entity-Relationship Diagram (ERD) with Keystore Boundary

12.2 Primary Table Specifications

Table 12.1 documents the primary server tables in Supabase PostgreSQL along with their schema attributes and security constraints.

Table 12.1: Supabase PostgreSQL Database Tables, Schemas, and Security Attributes

Table Name	Primary Key	Key Attributes & Types	Security Controls & RLS Policies
public.staff	id (UUID)	staff_code (UQ), name, email (UQ), department, designation, device, active, inside_campus	RLS Enabled. Public read for active staff; restricted admin update.
attendance_records	id (UUID)	staff_code, staff_name, department, date (DATE), day, time, status, location, latitude, longitude, device_id	RLS Enabled. Read restricted to session owner or admin. Idempotent daily write.

security_events	id (UUID)	time, staff, event, device (stores approval token, status, expires_at), location, result, severity	RLS Enabled. Immutable security audit log. Stores one-time face change tokens.
admin_users	id (VARCHAR)	username (UQ), name, email (UQ), role, password_hash, salt, active	Strict RLS: SELECT USING (false) for anon/public. Service role only.
admin_token_revocations	token_id (VARCHAR)	username, revoked_at, expires_at, reason	Strict RLS. Cleaned via clean_expired_admin_revocations() RPC.
devices	id (VARCHAR)	staff_id (FK), staff_code, model, os, status, registered_at	RLS Enabled. Tracks authorized physical Android smartphones.
geofence_zones	id (UUID)	name, campus_name, polygon_coords (TEXT), center_lat, center_lng, radius_meters, active	RLS Enabled. Stores authoritative 19-point polygon coordinates string.

CHAPTER 13

MATHEMATICAL FORMULATIONS & ALGORITHMS

13.1 Ray-Casting Jordan Curve Point-in-Polygon (PIP)

Containment inside the 19-vertex polygon is evaluated using the Jordan Curve theorem ray-casting algorithm. A horizontal semi-infinite ray is cast eastward from the test point (x, y) = (lng, lat). The algorithm counts the number of times the ray intersects the polygon's edges. An odd number of intersections indicates containment inside the polygon; an even number indicates the point lies outside. The time complexity is O(N) where N = 19 vertices, executing in < 5 microseconds.

```
FUNCTION isPointInPolygon(point, polygon):
  x = point.lng,  y = point.lat
  inside = FALSE
  FOR i = 0, j = polygon.length - 1; i < polygon.length; j = i++:
    xi = polygon[i].lng,  yi = polygon[i].lat
    xj = polygon[j].lng,  yj = polygon[j].lat
    IF xi == xj AND yi == yj: CONTINUE  // Skip closing segment C19 == C1
    intersect = ((yi > y) != (yj > y)) AND
                (x < (xj - xi) * (y - yi) / (yj - yi) + xi)
    IF intersect: inside = NOT inside
  RETURN inside
```

13.2 2D Constant-Velocity Kalman Filter for GNSS Smoothing

To mitigate multipath jitter from academic concrete structures, CampusAttend implements a linear 2D Kalman filter. WGS-84 coordinates are first transformed into local East/North Cartesian meters [x, y] relative to the campus centroid. The state vector is x = [x, y, vx, vy]^T. The prediction step applies a constant-velocity transition matrix F. Crucially, the measurement noise covariance R is dynamically scaled using the square of the raw GPS horizontal accuracy (varMeas = sigma^2), ensuring noisy readings (e.g. ±35m) are heavily dampened while precise fixes (e.g. ±4m) update state rapidly.

```
STATE VECTOR: x = [x (East_m), y (North_m), vx (m/s), vy (m/s)]^T
PREDICT STEP:
  x_pred = x + vx * dt,  y_pred = y + vy * dt
  P_pred = F * P * F^T + Q(q_acc, dt)
UPDATE STEP:
  sigma = max(raw_gps_accuracy_meters, 1.0)
  R = diag(sigma^2, sigma^2)
  S = H * P_pred * H^T + R
  K = P_pred * H^T * inv(S)
  x_upd = x_pred + K * (z_meas - H * x_pred)
  P_upd = (I - K * H) * P_pred
TRANSFORM: Convert [x_upd, y_upd] back to WGS-84 Lat/Lng for PIP evaluation
```

13.3 Umeyama 2D Similarity Transform & ArcFace Cosine Distance

Facial landmark normalization aligns 5 detected source points to standard ArcFace reference points using the Umeyama algorithm, computing rotation R, scale s, and translation t via Singular Value Decomposition (SVD). After extracting 512-dimensional feature vectors v1 (live face) and v2 (stored template), the similarity metric is Cosine Distance:

d_cosine = 1.0 - (v1 · v2) / (||v1||_2 * ||v2||_2)

Because vectors are L2-normalized during extraction (||v|| = 1.0), the formula reduces to d = 1.0 - sum(v1_i * v2_i). A match is confirmed if d <= 0.45 and separation margin >= 0.08.

13.4 Eye Aspect Ratio (EAR) Temporal Blink Liveness

Liveness verification tracks 6 landmark coordinates per eye [p1, p2, p3, p4, p5, p6]. The Eye Aspect Ratio is defined as:

EAR = (||p2 - p6|| + ||p3 - p5||) / (2 * ||p1 - p4||)

When an eye is open, EAR ranges between 0.28 and 0.35. During a natural blink, EAR rapidly dips below 0.235 for 1 to 15 frames (100ms – 400ms) before returning to the open state (>0.26). Static photograph attacks produce a constant EAR, failing liveness.

CHAPTER 14

USER INTERFACE & SCREEN FLOW

14.1 Mark Attendance Screen (/mark-attendance)

The primary staff interface provides real-time verification indicators across all presence factors: (1) Campus Network Card displaying connected SSID, IP address, and HMAC token status; (2) GPS Geofence Card displaying live latitude, longitude, horizontal accuracy, Kalman status, and interactive campus map; (3) Biometric Face Card with live camera preview, landmark tracking mesh, and blink prompt; and (4) Attendance Decision Action Button that unlocks only when all factors satisfy requirements.

14.2 Face Enrollment Console (/face-enrollment)

Allows staff members to enroll 1 to 5 facial reference images. If the profile is permanently locked, the interface displays a lock badge and presents the 'Request Face Change' modal, prompting the user for an administrative justification.

14.3 Administrative Governance Consoles

Administrative screens (/admin/staff, /admin/attendance, /admin/face-requests) provide complete oversight: inspecting registered staff accounts, filtering attendance records by department and date, reviewing security events, and approving face change requests with one-click 24-hour token issuance.

CHAPTER 15

TESTING AND EMPIRICAL VALIDATION

15.1 Deterministic Test-Case Execution Matrix

Table 15.1 documents the test execution matrix comprising 15 deterministic unit and integration test suites executed against the CampusAttend codebase. Every test case is grounded in actual repository test files and logs.

Table 15.1: Empirical Test-Case Execution Matrix: Deterministic Unit and Integration Tests

Test ID	Feature Tested	Input / Scenario	Expected Result	Actual Result	Status	Evidence Source
TC-01	Kalman Coordinate Precision	Round-trip Lat/Lng to local meters	0.0000 mm roundtrip error	0.0000 mm error verified	PASS	test-kalman-filter.js
TC-02	Kalman Jitter Smoothing	Raw GPS jitter variance 1.49m	Smoothed to < 0.50m	Smoothed to 0.41m (72% dampening)	PASS	test-kalman-filter.js
TC-03	Kalman Outlier Suppression	155m noisy spike with ±85m accuracy	Spike heavily dampened	Filtered pos offset: 0.3m (99.8% dampening)	PASS	test-kalman-filter.js
TC-04	GPS Accuracy Cutoff 20.1m	Horizontal accuracy = ±20.1m	isAccepted === false	Rejected: GPS ACCURACY INSUFFICIENT	PASS	test-gps-accuracy-suite.js
TC-05	GPS Accuracy Cutoff 20.0m	Horizontal accuracy = ±20.0m	isAccepted === true	Accepted: GPS ACCURACY ACCEPTED	PASS	test-gps-accuracy-suite.js
TC-06	GPS Target Goal 9.9m	Horizontal accuracy = ±9.9m	isTargetReached === true	Accepted: GPS ACCURACY EXCELLENT (<10m)	PASS	test-gps-accuracy-suite.js
TC-07	Geofence Inside Point	Centroid (11.676982, 78.125354)	PIP: INSIDE	INSIDE (91.8m to boundary)	PASS	test-gps-accuracy-suite.js
TC-08	Geofence Outside Point	North Highway (11.685000, 78.125300)	PIP: OUTSIDE	OUTSIDE (649.9m to boundary)	PASS	test-gps-accuracy-suite.js
TC-09	Wi-Fi 'sona' Case-Insensitive	SSIDs: SONA-WIFI, sona-wifi, SoNa	isAuthorized === true	All authorized (46/46 passed)	PASS	test-unified-wifi-policy.js
TC-10	Wi-Fi Unauthorized SSID	SSIDs: Home_WiFi, CollegeNet, m	isAuthorized === false	Strictly rejected	PASS	test-unified-wifi-policy.js
TC-11	Wi-Fi HMAC Token Verification	Valid token with SSID SONA-WIFI	Token verified (HTTP 200)	Token valid, payload verified	PASS	test-unified-wifi-policy.js
TC-12	Wi-Fi Tampered Token	Modified signature byte	Token rejected	INVALID_SIGNATURE returned	PASS	test-unified-wifi-policy.js
TC-13	Staff Face Permanent Lock	Attempt repeat enrollment without token	Rejected with ALREADY_ENROLLED	Strictly rejected; templates intact	PASS	test-staff-face-lock-workflow.js
TC-14	Admin 24h Approval Token	Approval token expires_at in past	Rejected with EXPIRED_TOKEN	HTTP 403 EXPIRED_APPROVAL_TOKEN	PASS	test-staff-face-lock-workflow.js
TC-15	Atomic Concurrency RPC	Concurrent saves with same token	Exactly 1 succeeds, 1 rejected	HTTP 200 (first) and HTTP 403	PASS	test-staff-face-lock-

				(second)		workflow.js
TC-16	Client Zero Biometrics Audit	Client source code check for cloud post	Zero cloud biometric posts	Assertion error: legacy call present	FAIL*	test-staff-face-local-enrollment.js
TC-17	Physical Multi-Device Isolation	Physical device test on 5 OEM phones	Verified on physical hardware	Pending live physical testbed	NOT VERIFIED	Pending Physical Device Test

15.2 Live Production Vercel Deployment Verification

Table 15.2 details the results of running the live deployment verification suite (verify-production-live.js) against the active production URL: https://geo-fenced-attendence-system.vercel.app.

Table 15.2: Live Vercel Production Deployment Verification Suite Outcomes

Test Step	Live Target Endpoint	Scenario Evaluated	HTTP Code	Observed Outcome	Status
1	POST /api/staff/register	Missing shared authorization code	403 Forbidden	Rejected: Shared password required	PASS
2	POST /api/staff/register	Invalid shared authorization code	403 Forbidden	Rejected: Invalid shared password	PASS
3	POST /api/staff/register	Password mismatch in registration	400 Bad Request	Rejected: Passwords do not match	PASS
4	POST /api/staff/register	Valid registration with staff@123	201 Created	Staff SCT-LIVE-4223 created in Supabase	PASS
5	POST /api/staff/register	Duplicate staff ID registration	409 Conflict	Rejected: Staff ID already registered	PASS
6	Supabase PostgreSQL Audit	Inspect stored password in database	Direct SQL	Stored as salted PBKDF2 hash (Zero plaintext)	PASS
7	POST /api/staff/login	Valid login with personal credentials	200 OK	Authenticated; issued HMAC token & cookie	PASS
8	POST /api/staff/login	Invalid password login attempt	401 Unauthorized	Rejected: Invalid staff credentials	PASS
9	POST /api/admin/login	Admin login with username moulish	200 OK	Admin authenticated; role strictly 'admin'	PASS
10	POST /api/attendance	Missing network authorization token	403 Forbidden	CAMPUS_NETWORK_UNAUTHORIZED	PASS
11	POST /api/attendance	GPS coordinates outside polygon	403 Forbidden	GPS_OUTSIDE_CAMPUS rejected	PASS
12	POST /api/face/verify	Cloud biometric vector matching	200 OK	CLOUD_BIOMETRICS_DISABLED (Device-Only Mode)	PASS

CHAPTER 16

DEPLOYMENT, INSTALLATION & DEVOPS

16.1 Production Web Deployment on Vercel

The web application and serverless API endpoints are deployed on Vercel Edge Runtime. Active Production URL: https://geo-fenced-attendence-system.vercel.app Mark Attendance Route: https://geo-fenced-attendence-system.vercel.app/mark-attendance The deployment leverages Vercel's global CDN for edge delivery of static assets (HTML, CSS, JS, ONNX model weights), while dynamically routing API requests to serverless Node.js lambda functions.

16.2 Supabase Cloud Configuration

The PostgreSQL database is provisioned on Supabase Cloud: Project URL: https://qvjcxoznvhoagclbyhad.supabase.co The database enables pgcrypto, uuid-ossp, and vector extensions. Database tables and security policies are provisioned using declarative SQL scripts in supabase/schema.sql and versioned migrations in supabase/migrations/.

16.3 Android APK Build and Distribution Process

The Android APK is compiled using Gradle 8.5 with Kotlin 1.9.22 targeting Android 14 (API Level 34):

- Build Command: ./gradlew assembleDebug
- Output Binary: android/app/build/outputs/apk/debug/app-debug.apk
- Package Name: com.campusattend.biometric
- Binary Size: ~154.6 MB (Includes bundled w600k_mbf.onnx model and native ONNX Runtime C++ shared libraries libonnxruntime.so for ARM64 and x86_64).

16.4 Environment Variables & Secret Configuration

The project requires the following environment variables (configured in .env and Vercel Project Settings):

- SUPABASE_URL: URL of the Supabase PostgreSQL instance.
- SUPABASE_ANON_KEY: Publishable client key for authenticated queries.
- SUPABASE_SERVICE_ROLE_KEY: Privileged backend key for administrative RPC execution.
- CAMPUS_AUTH_SECRET: Secret key for signing and verifying HMAC-SHA256 network tokens.
- STAFF_AUTH_SECRET: Secret key for signing staff session tokens.
- CAMPUS_ADMIN_SECRET: Secret key for administrator session tokens.
- AUTHORIZED_CAMPUS_EGRESS_IPS: Comma-separated list of campus public egress IPs (111.92.42.18, 115.247.87.98).

16.5 Independent Updating of Web App and Android APK

A key architectural advantage of the hybrid design is independent lifecycle deployment:

- Web Client Updates: UI enhancements, administrative reports, styling changes, and serverless API updates deploy instantly to Vercel without requiring users to download new APK binaries.
- Android APK Updates: The APK requires re-compilation and redistribution only when modifying low-level native bridge interfaces, CameraX lifecycle hooks, or Android Keystore cryptographic algorithms.

CHAPTER 17

RESULTS AND TECHNICAL DISCUSSION

17.1 Implementation Verification Matrix

Table 17.1 provides an authoritative categorization of project capabilities, separating features verified by unit tests, features verified by live deployment checks, features confirmed by code inspection, and features pending physical device tests.

Table 17.1: Verified Implementation Status Matrix: Implemented vs. Pending Hardware Checks

Feature / Architectural Component	Verification Method	Empirical Outcome / Evidence Source	Current Verification Status
2D Kalman Filter Stabilization	Unit Test Suite	10/10 tests passed; 0.0000 mm roundtrip error; 99.8% outlier suppression	VERIFIED BY TESTS
GPS Accuracy Gating (20.0m / 9.9m)	Integration Test Suite	Exact threshold gating verified across 20.1m, 20.0m, 10.0m, 9.9m	VERIFIED BY TESTS
19-Point Polygon PIP Containment	Geometric Benchmark	100% correct containment across inside/outside coordinates	VERIFIED BY TESTS
Unified Wi-Fi 'sona' Gating	Automated Security Suite	46/46 passed; case-insensitive SSID matching & HMAC token verified	VERIFIED BY TESTS
Staff Self-Registration API	Live Vercel Production	HTTP 201 Created with staff@123; duplicate rejection HTTP 409	VERIFIED LIVE ON VERCEL
PBKDF2 Password Hashing	Live Database Audit	Stored as salted PBKDF2 hash in Supabase Cloud (Zero plaintext)	VERIFIED LIVE ON VERCEL
Staff Login & HMAC Cookie	Live Vercel Production	HTTP 200 OK; issued HMAC staff_session token & HttpOnly cookie	VERIFIED LIVE ON VERCEL
Admin Login (username moulish)	Live Vercel Production	HTTP 200 OK; role strictly verified as 'admin'	VERIFIED LIVE ON VERCEL
Permanent Face Lock & 24h Token	Security Test Suite	99/100 passed; token consumption & atomic RPC locking verified	VERIFIED BY TESTS
Zero Cloud Biometric Storage	API Code & Endpoint Audit	api/face/verify returns CLOUD_BIOMETRICS_DISABLED (Device-Only Mode)	VERIFIED BY CODE INSPECTION
Legacy Client Call Cleanup	Client Source Code Audit	submitStaffFaceEnrollment call still present in face-enrollment.tsx	INCONSISTENCY IDENTIFIED
Multi-OEM Physical Hardware Run	Real-World Field Test	Requires testing across 5+ physical Android OEM phones (Samsung, Xiaomi)	PENDING PHYSICAL DEVICE TEST

17.2 Discussion of Empirical Findings

The empirical findings demonstrate that multi-sensor cross-validation successfully resolves the long-standing vulnerabilities of single-factor attendance systems. While GPS alone can be spoofed via mock location software, spoofing is rendered ineffective because the server concurrently demands an HMAC token derived from native Wi-Fi SSID inspection. Similarly, an attacker connecting to a portable hotspot named 'sona-wifi' outside campus is blocked because their GNSS coordinates evaluate to OUTSIDE the 19-point polygon. Finally, static photograph presentation attacks are thwarted by temporal EAR blink verification.

CHAPTER 18

PRACTICAL ADVANTAGES & LIMITATIONS

18.1 Practical Advantages

- **Elimination of Terminal Queues:** Eliminates morning entrance bottlenecks by allowing hundreds of faculty members to mark attendance concurrently from anywhere within the 19-point campus perimeter.
- **Uncompromised Biometric Privacy:** Facial photographs and 512-D embedding vectors remain 100% on the staff member's device in Android Keystore, eliminating centralized biometric data breach liability.
- **Zero Dedicated Hardware Expenditure:** Requires zero dedicated optical scanning hardware, badge readers, or physical cabling, saving substantial capital expenditure.
- **Robust Anti-Proxy Enforcement:** Multi-sensor cross-validation (Wi-Fi + GPS + Face) eliminates proxy attendance, buddy punching, and remote check-ins.

18.2 Practical Limitations

- **Indoor Satellite Attenuation:** GNSS satellite signals suffer severe attenuation inside subterranean basements or heavily reinforced concrete lecture halls, requiring staff to acquire a fix near windows or outdoor walkways.
- **Device-Bound Re-enrollment Overhead:** Because biometric templates are cryptographically bound to hardware device IDs, staff members must re-enroll their facial templates under administrative oversight when replacing their phone.
- **Browser Platform Sandboxing:** Standard desktop and mobile web browsers cannot access Wi-Fi SSIDs due to W3C sandboxing, requiring the native Android APK for full Factor 1 verification.

CHAPTER 19

FUTURE ENHANCEMENTS

The following prospective enhancements are identified for future development iterations (strictly labeled as proposed improvements):

- **1. Google Play Integrity API Hardware Attestation:** Incorporate Google Play Integrity API (formerly SafetyNet) to verify device genuineness, detect root/Magisk binaries, identify locked bootloaders, and confirm execution in an untampered environment.
- **2. Indoor BLE Micro-Location Beacons:** Deploy battery-powered Bluetooth Low Energy (BLE) iBeacon / Eddystone transmitters inside indoor laboratories and basements to provide micro-location presence verification where satellite GNSS signals are unavailable.
- **3. Signed Offline Attendance Queuing:** Implement a cryptographically signed offline attendance queue using local SQLite storage, allowing staff to mark presence during network outages and sync records upon reconnection.
- **4. Enterprise HRMS & ERP Synchronization:** Develop bidirectional REST webhooks integrating CampusAttend with enterprise institutional HRMS and ERP platforms (SAP, Oracle PeopleSoft, TCS iON) for automated payroll integration.

CHAPTER 20

CONCLUSION

The CampusAttend project demonstrates that robust, enterprise-grade institutional attendance management can be achieved without sacrificing employee privacy or incurring heavy capital expenditures on fixed biometric hardware. By combining a 19-point authoritative campus polygon geofence with 2D Kalman filter stabilization, native Android Wi-Fi SSID authentication, on-device InsightFace MobileFaceNet ArcFace facial recognition, temporal EAR blink liveness verification, and Android Keystore AES-256-GCM hardware encryption, CampusAttend delivers an uncompromising, multi-factor physical presence verification solution.

Empirical testing confirms deterministic coordinate conversion precision, 99.8% outlier jitter suppression, 100% geofence containment accuracy, and reliable zero-client-trust serverless validation across active Vercel and Supabase production environments. By enforcing strict device-local biometric isolation, CampusAttend provides a scalable, privacy-preserving architectural blueprint suitable for modern academic institutions and decentralized enterprise campuses.

REFERENCES

[1] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, 2019, pp. 4690-4699.

[2] S. Umeyama, "Least-Squares Estimation of Transformation Parameters Between Two Point Patterns," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 13, no. 4, pp. 376-380, April 1991.

[3] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," Journal of Basic Engineering, vol. 82, no. 1, pp. 35-45, March 1960.

[4] T. Soukupová and J. Čech, "Real-Time Eye Blink Detection using Facial Landmarks," in 21st Computer Vision Winter Workshop (CVWW), Rimske Toplice, Slovenia, February 2016.

[5] National Geospatial-Intelligence Agency (NGA), "World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems," NGA Standardization Document NGA.STND.0036_1.0.0_WGS84, 2014.

[6] Android Open Source Project (AOSP), "Hardware-backed Keystore and Keymaster Architecture," Google Developers Documentation, 2024. [Online]. Available: https://source.android.com/security/keystore

[7] Microsoft Corporation, "ONNX Runtime: Cross-Platform, High Performance ML Inferencing and Training Accelerator," 2024. [Online]. Available: https://onnxruntime.ai

[8] Google LLC, "MediaPipe Solutions: On-Device Vision Tasks Guide," Google AI Developers Documentation, 2024. [Online]. Available: https://developers.google.com/mediapipe

[9] Supabase Inc., "PostgreSQL and pgvector: Vector Similarity Search Documentation," 2024. [Online]. Available: https://supabase.com/docs/guides/database/extensions/pgvector

[10] Vercel Inc., "Vercel Edge Functions and Serverless Runtimes," Vercel Documentation, 2024. [Online]. Available: https://vercel.com/docs/functions

[11] Ministry of Law and Justice, Government of India, "The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023)," The Gazette of India, New Delhi, August 2023.

[12] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation - GDPR)," Official Journal of the European Union, April 2016.

APPENDIX A: PROJECT REPOSITORY STRUCTURE & KEY FILES

The CampusAttend codebase is organized into modular frontend, backend serverless, native Android, and database migration directories:

```
geo-fenced-attendence-system/
├── android/                        # Android Native Kotlin Application (API 34)
│   └── app/src/main/java/com/campusattend/biometric/
│       ├── bridge/                 # AndroidBiometricBridge.kt (Keystore + Room bridge)
│       ├── location/               # GeofenceManager.kt (19-pt polygon), GpsKalmanFilter.kt
│       ├── recognition/            # FaceEmbeddingService.kt (ArcFace ONNX), Preprocessing
│       ├── security/               # BiometricKeystoreManager.kt (AES-256-GCM TEE)
│       ├── wifi/                   # AndroidWifiBridge.kt (SSID 'sona' inspection)
│       └── data/                   # Room AppDatabase.kt, EncryptedFaceTemplateEntity.kt
├── api/                            # Vercel Serverless Function Endpoints
│   ├── attendance.ts               # POST /api/attendance (Zero-trust 3-factor gating)
│   ├── wifi-status.ts              # POST /api/wifi-status (SSID validation & HMAC token)
│   ├── face/verify.ts              # POST /api/face/verify (Device-only privacy enforcement)
│   ├── staff/                      # Staff login, registration, and face endpoints
│   └── admin/                      # Administrator authentication, staff CRUD, approvals
├── src/                            # TanStack / React 19 Frontend Web Application
│   ├── routes/                     # TanStack Router pages (mark-attendance, face-enrollment)
│   ├── lib/geofence/               # geofence-service.ts (19-pt polygon), kalman-filter.ts
│   ├── lib/face-recognition/       # arcface-engine.ts, blink-detection.ts, face-config.ts
│   └── lib/                        # native-biometric-bridge.ts, staff-auth.ts, wifi-config.ts
├── supabase/                       # Supabase Database Schemas and Migrations
│   ├── schema.sql                  # Primary PostgreSQL tables, RLS policies, seeds
│   └── migrations/                 # Atomic token consumption RPC, admin revocation table
├── scripts/                        # Empirical Test Suites and Live Verification Harnesses
│   ├── test-kalman-filter.js       # 2D Kalman filter deterministic unit test suite
│   ├── test-gps-accuracy-suite.js  # GPS accuracy cutoff & polygon containment matrix
│   ├── test-unified-wifi-policy.js # Wi-Fi SSID validation & HMAC token security suite
│   ├── test-staff-face-lock-...    # Permanent lock & atomic RPC verification suite
│   └── verify-production-live.js   # Live Vercel HTTPS production verification harness
├── public/models/                  # w600k_mbf.onnx (ArcFace MobileFaceNet weights)
├── package.json                    # Project dependencies, scripts, and build metadata
└── vercel.json                     # Vercel serverless routing and headers configuration
```

APPENDIX B: DATABASE DDL SCHEMA EXCERPTS (SUPABASE POSTGRESQL)

```
-- 1. Staff Table (Metadata & Active Status)
CREATE TABLE IF NOT EXISTS public.staff (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    staff_code VARCHAR(64) UNIQUE NOT NULL,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    department VARCHAR(255) NOT NULL,
    designation VARCHAR(255) NOT NULL,
    active BOOLEAN DEFAULT true NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- 2. Attendance Records Table (Immutable Receipts)
CREATE TABLE IF NOT EXISTS public.attendance_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    staff_code VARCHAR(64) NOT NULL,
    staff_name VARCHAR(255) NOT NULL,
    department VARCHAR(255) NOT NULL,
    date DATE DEFAULT CURRENT_DATE NOT NULL,
    day VARCHAR(32) NOT NULL,
    time VARCHAR(32) NOT NULL,
    status VARCHAR(32) DEFAULT 'Present' NOT NULL,
    location VARCHAR(255) DEFAULT 'Main Campus',
    verification VARCHAR(32) DEFAULT 'Verified' NOT NULL,
    latitude DOUBLE PRECISION,
    longitude DOUBLE PRECISION,
    device_id VARCHAR(64),
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- 3. Atomic Face Approval Consumption RPC (Zero Cloud Vectors)
CREATE OR REPLACE FUNCTION public.atomic_consume_face_approval_token(
    p_staff_code VARCHAR, p_token VARCHAR, p_device_id VARCHAR
) RETURNS JSONB LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE
    v_req_id UUID; v_device VARCHAR; v_now TIMESTAMPTZ := NOW();
BEGIN
    SELECT id, device INTO v_req_id, v_device FROM public.security_events
    WHERE UPPER(staff) = UPPER(TRIM(p_staff_code)) AND result = 'Allowed'
      AND device LIKE ('%token:' || TRIM(p_token) || '%')
      AND device LIKE '%status:approved%' AND device NOT LIKE '%status:consumed%'
    FOR UPDATE; -- Exclusive row lock serializes concurrent requests
    IF v_req_id IS NULL THEN
        RETURN jsonb_build_object('success', false, 'error', 'Invalid or consumed token');
    END IF;
    UPDATE public.security_events
    SET device = 'token:' || TRIM(p_token) || '|status:consumed|device_id:' || p_device_id
    WHERE id = v_req_id;
    RETURN jsonb_build_object('success', true, 'isLocked', true);
END; $$;
```

APPENDIX C: SERVERLESS API ENDPOINT REFERENCE

Table C.1: CampusAttend Serverless API Endpoint Specifications

HTTP Method & Path	Auth Required	Key Parameters / Payload	Response Summary & Success Codes
POST /api/staff/register	Shared Code (staff@123)	staff_code, name, department, password, registrationCode	HTTP 201 Created. PBKDF2 hash stored in staff table.
POST /api/staff/login	None (Credentials)	staffId / staff_code, password	HTTP 200 OK. Issues HMAC staff_session token in HttpOnly cookie.
POST /api/admin/login	None (Admin Creds)	username, password	HTTP 200 OK. Issues HMAC admin_session token in HttpOnly cookie.
POST /api/wifi-status	None (Device info)	isNativeBridge, ssid, transport, isWifi	HTTP 200 OK. Mints HMAC networkAuthToken if SSID contains 'sona'.
POST /api/face/verify	None (Vector check)	descriptor / embedding (512 floats)	HTTP 200 OK. Code CLOUD_BIOMETRICS_DISABLED (Device-Only Mode).
GET /api/staff/face/status	Staff Session	None (derived from session)	HTTP 200 OK. Returns isLocked, sampleCount, pendingRequest.
POST /api/staff/face/enroll	Staff Session	token (if approved), deviceId	HTTP 201 Created. Initial lock or calls atomic RPC for replacement.
POST /api/staff/face/request-change	Staff Session	reason, stagedCount	HTTP 201 Created. Submits change request to security_events.
POST /api/attendance	Staff / Attestation	networkAuthToken, latitude, longitude, accuracy, biometricAttestation	HTTP 200 OK. 3-Factor validated; writes attendance receipt.
GET /api/attendance	Staff / Admin Session	staff_code, date, department, limit	HTTP 200 OK. Returns filtered attendance history records.

APPENDIX D: COMPREHENSIVE VIVA VOCE PREPARATION (30 Q&A)

This section provides 30 project-specific questions and authoritative technical answers covering all aspects of the CampusAttend implementation, tailored for viva voce examinations, project review presentations, and technical evaluation panels.

Q1: What is the primary objective of CampusAttend, and what problems does it solve?

CampusAttend is a secure biometric attendance management system that replaces fixed wall-mounted biometric appliances with a mobile-native architecture. It eliminates entrance queue bottlenecks, prevents location spoofing through multi-sensor cross-validation (Wi-Fi + GPS + Face), and preserves employee privacy by storing biometric data exclusively on-device.

Q2: Why is a 19-point polygon used instead of a circular radius for geofencing?

A circular radius centered at a campus midpoint is radially symmetric and inevitably covers exterior public roads, private residences, and off-campus cafes where unauthorized attendance could be marked. An authoritative 19-point polygon precisely conforms to the complex physical boundary of Sona College of Technology, enclosing only authorized academic, laboratory, and residential grounds.

Q3: What mathematical algorithm evaluates whether a coordinate is inside the campus polygon?

The Jordan Curve theorem ray-casting point-in-polygon algorithm. It casts a horizontal semi-infinite ray eastward from the user's coordinate. By counting intersections with polygon edge segments, an odd count proves containment inside the polygon, while an even count proves the coordinate lies outside. It executes in O(N) time (under 5 microseconds for 19 vertices).

Q4: Why does the system need a 2D Kalman filter for GPS coordinates?

Urban collegiate environments feature multi-story concrete academic blocks that cause GNSS multipath reflections, resulting in rapid positional jitter (artificial wandering of ±5m to ±25m even while standing still). The 2D Kalman filter models motion as constant velocity and dynamically dampens jitter by up to 99.8% based on raw measurement variance.

Q5: Does the Kalman filter alter or fabricate the device's reported GPS horizontal accuracy?

No. The Kalman filter converts WGS-84 coordinates to local Cartesian tangent-plane meters to smooth latitude and longitude coordinates. It strictly preserves the raw, unmodified horizontal accuracy reported by the device GNSS receiver. The system never fabricates a high-accuracy fix.

Q6: What is the mandatory GPS accuracy cutoff for marking attendance?

±20.0 meters. Any GPS reading with horizontal accuracy uncertainty greater than 20.0m is strictly rejected with the status 'GPS ACCURACY INSUFFICIENT'. When accuracy is ≤ 20.0m, attendance proceeds immediately, while the filter continues striving toward the optimization target goal of < 10.0m.

Q7: How does Wi-Fi network verification work in CampusAttend?

The native Android APK queries the active network via WifiManager. If the connected SSID contains 'sona' (case-insensitive substring match), the client requests a cryptographically signed HMAC-SHA256 network authorization token from the server with a 5-minute time-to-live (TTL).

Q8: What happens if a staff member attempts to mark attendance using mobile cellular data?

The AndroidWifiBridge identifies the network transport as 'cellular'. The system rejects the request with code 'CELLULAR_DATA' and status 'FINGERPRINT_CHECK_FAILED', instructing the user to connect to an authorized campus Wi-Fi network.

Q9: What happens if an employee sets up a mobile hotspot named 'sona-wifi' outside campus?

While the hotspot satisfies the Wi-Fi SSID rule (Factor 1), the employee's physical GNSS coordinates will evaluate to OUTSIDE the 19-point campus polygon (Factor 2 fails). Because attendance requires all three factors simultaneously, the server rejects the request with HTTP 403.

Q10: What neural network model is used for face recognition, and what descriptor does it generate?

InsightFace MobileFaceNet ArcFace (w600k_mbf.onnx), trained on MS1M-w600k. It takes a 112×112 aligned facial crop and outputs a 512-dimensional floating-point embedding vector normalized to unit L2 norm (||v|| = 1.0).

Q11: How is the face cropped and aligned before feature extraction?

MediaPipe / SSD MobileNet detects 5 facial landmarks (eyes, nose, mouth corners). The Umeyama algorithm computes a 2D affine/similarity transformation matrix mapping these landmarks to standard ArcFace reference points, warping the crop into an aligned 112×112 RGB image.

Q12: What metric and threshold are used for facial similarity matching?

Cosine Distance: d = 1.0 - (v1 · v2). Because vectors are L2-normalized, this equals 1.0 minus the dot product. A match is confirmed if d <= 0.45 (equivalent to Cosine Similarity >= 0.55), with a minimum separation margin of >= 0.08 from the second-best candidate.

Q13: How does the system prevent spoofing via printed photographs or smartphone screens?

Through multi-layer temporal liveness detection, primarily Eye Aspect Ratio (EAR) blink tracking. An eye blink causes EAR to dip below 0.235 for 1 to 15 frames before opening (>0.26). Static photographs exhibit constant EAR, failing the required challenge.

Q14: Where are facial images and biometric embedding vectors stored?

Exclusively on the staff member's personal Android smartphone. Templates are encrypted inside the Android Keystore Trusted Execution Environment (TEE) and stored in local Room SQLite storage. Biometrics are NEVER uploaded to Supabase, PostgreSQL, or backend servers.

Q15: How does Android Keystore protect biometric templates from extraction?

The master AES-256 key is generated inside secure hardware (TEE/StrongBox) and cannot be exported by software. Templates are encrypted with AES-256 in Galois/Counter Mode (GCM). Authenticated Additional Data (AAD) is set to '$staffId:$deviceId', cryptographically binding the ciphertext to that specific user and phone.

Q16: What happens if an attacker copies the SQLite database file to another phone?

Decryption fails immediately. Because the Android device ID is embedded in the GCM Authenticated Additional Data (AAD), the hardware crypto chip detects a security tag mismatch and throws AEADBadTagException, refusing to decrypt the template.

Q17: How does the biometric profile lock and admin approval workflow function?

Upon initial enrollment (1-5 photos), the profile is permanently locked (isLocked: true). To change facial data, staff submit a change request with a justification. An administrator reviews it on /admin/face-requests and approves it, issuing a single-use 24-hour token (ftok_...). The staff member then activates the new template on their phone, which consumes the token and immediately re-locks the profile.

Q18: What is the role of the PostgreSQL RPC atomic_consume_face_approval_token?

It ensures atomic, concurrency-safe token consumption. It validates that the token exists, is approved, and has not expired (fail-closed TIMESTAMPTZ check). It updates the status to 'consumed' within a single transaction, preventing replay attacks.

Q19: Why does the atomic consumption RPC use SELECT ... FOR UPDATE?

Row-level locking with FOR UPDATE serializes concurrent requests. If an attacker submits two simultaneous HTTP requests with the same approval token, the second request is forced to wait for the first to complete, after which it sees status:consumed and terminates with HTTP 403 Forbidden.

Q20: What is 'Zero Client Trust' and how is it implemented?

Zero Client Trust means the server ignores all client-asserted verification booleans. Instead, the client must submit cryptographic proofs: an HMAC network token for Wi-Fi, raw coordinates re-evaluated by the server for GPS, and an HMAC biometric attestation token for face recognition.

Q21: How does the server prevent attendance attestation replay attacks?

The biometric attestation token issued by the Android Keystore embeds a millisecond UNIX timestamp. The server enforces a strict 120-second validity window. An intercepted token replayed after 2 minutes is rejected as expired.

Q22: How is duplicate attendance prevented?

Upon receiving a valid attendance payload, the server checks the attendance_records table for an existing record matching the staff_code and the current date in Asia/Kolkata timezone. If found, it returns the existing receipt with alreadyMarked: true, performing zero duplicate writes.

Q23: What database schema is used on the server?

Supabase PostgreSQL 15. Key tables include staff (account metadata), attendance_records (immutable check-in receipts), security_events (audit log & approval tokens), admin_users (admin accounts), and admin_token_revocations (logout revocation registry).

Q24: Why does POST /api/face/verify return CLOUD_BIOMETRICS_DISABLED?

To enforce the architectural privacy guarantee. The endpoint intentionally disables cloud vector matching and returns code 'CLOUD_BIOMETRICS_DISABLED', ensuring that face recognition is performed strictly on-device in the Android Keystore.

Q25: What happens if a staff member opens /mark-attendance in standard desktop Google Chrome?

Web browsers cannot access Wi-Fi SSIDs due to W3C sandboxing. The Wi-Fi card reports code 'BROWSER_SSID_UNAVAILABLE', informing the user that Factor 1 requires the native Android APK.

Q26: How are staff passwords secured in the database?

Using PBKDF2 with a unique 16-byte cryptographically secure random salt and 10,000 iterations. Passwords are never stored in plaintext.

Q27: How does administrator logout revocation work?

When an admin logs out, the server extracts the token ID or signature hash and inserts it into public.admin_token_revocations with an expiration time. Future requests presenting that token are rejected even if the cryptographic signature is valid.

Q28: What is the production URL, and how was deployment verified?

https://geo-fenced-attendence-system.vercel.app. Verified using an automated test suite (verify-production-live.js) confirming 32 live test assertions including staff registration, PBKDF2 hashing, login, admin authentication, and attendance gating.

Q29: What known inconsistencies or partial implementations were identified in the codebase?

A legacy function call to submitStaffFaceEnrollment remains in src/routes/face-enrollment.tsx (even though backend APIs reject cloud vectors), and multi-OEM physical hardware testing on varied consumer devices remains pending field deployment.

Q30: What are the key realistic limitations of the system?

GNSS satellite signal attenuation inside basements; device-bound re-enrollment overhead when a user switches phones; and the requirement of the native Android APK for Wi-Fi SSID reading due to browser sandbox restrictions.

APPENDIX E: TECHNICAL GLOSSARY

ArcFace (Additive Angular Margin Loss): A deep convolutional facial recognition architecture that introduces an additive angular margin penalty directly into the target sphere angle, maximizing inter-class variance and minimizing intra-class feature distance.

Authenticated Additional Data (AAD): An AEAD cryptographic input authenticated by the cipher tag without being encrypted, ensuring ciphertext cannot be decrypted under a different context (e.g. different user or device).

Eye Aspect Ratio (EAR): A scalar metric computed from 6 facial landmarks surrounding the ocular palpebral fissure, quantifying eye openness for temporal blink tracking.

Fused Location Provider: An Android location client that intelligently combines satellite GNSS, cellular triangulation, and Wi-Fi signals to produce optimal horizontal coordinates.

Jordan Curve Theorem: A topological theorem stating that every continuous non-self-intersecting closed curve divides a plane into an 'inside' and an 'outside' region, forming the foundation of ray-casting point-in-polygon tests.

Kalman Filter: An optimal recursive algorithm that estimates the true hidden state of a dynamic system from a series of noisy measurements observed over time.

Row-Level Security (RLS): A PostgreSQL security subsystem that restricts data rows accessible to specific SQL operations based on evaluated security predicates and user roles.

Trusted Execution Environment (TEE): A hardware-isolated secure enclave within modern mobile processor SoCs providing hardware-level key isolation and cryptographic acceleration.